

JAVA PROJEKT

Matrizenrechner mit MVC und GUI

Objektorientierte Programmierung mit Java

Addition, Subtraktion, Multiplikation, Transponieren, Determinante, Inverse

In diesem Projekt:

- Aufbau eines Matrizenrechners nach dem **MVC-Prinzip**
- Implementierung einer **GUI** mit Java Swing
- Rechenoperationen für Matrizen
- Determinante bis **5x5**
- Hilfsmethode zum Streichen von Zeilen und Spalten
- Inverse, falls die Matrix invertierbar ist

Inhaltsverzeichnis

1	Einleitung	3
2	Ziel des Projekts	3
3	Anforderungen	3
3.1	Funktionale Anforderungen	4
3.2	Nicht-funktionale Anforderungen	4
3.3	Mathematische Anforderungen	4
3.4	Fehlerbehandlung und Exceptions	4
3.5	Zusammenfassung	5
4	MVC-Prinzip	5
4.1	Model	5
4.2	View	5
4.3	Controller	5
4.4	Vorteile des MVC-Prinzips	6
5	Klassenstruktur	6
5.1	Matrix	6
5.2	MatrixException	6
5.3	MatrixView	7
5.4	MatrixController	7
5.5	Zusammenfassung	7
6	Matrix-Klasse	7
6.1	Attribute und Konstruktoren	8
6.1.1	Konstruktoren	8
6.1.2	Bedeutung der Konstruktoren	9
6.2	Methodenübersicht	9
6.2.1	Zugriff auf Matrixelemente	10
6.2.2	Addition	11
6.2.3	Subtraktion	12
6.2.4	Multiplikation	12
6.2.5	Transponieren	13
6.2.6	Spur	14
6.2.7	Hilfsmethode zur Erzeugung einer Untermatrix	15
6.2.8	Determinante	16
6.2.9	Vorzeichenfunktion	17
6.2.10	Adjunkte	17
6.2.11	Inverse	18
6.2.12	toString()	19
6.2.13	equals() und hashCode()	20
6.3	Resümee zum Model	21
7	Exceptions	21
8	Controller	22
8.1	Controller-Umsetzung in Java	23
8.2	Controller-Umsetzung mit Lambda-Ausdrücken	26
8.3	Resümee zum Controller	28

9 View	28
9.1 Bestandteile der Oberfläche	29
9.2 Zusammenarbeit mit Controller und Model	29
9.3 View-Umsetzung in Java	30
9.4 Resümee zur View	33
10 Main-Klasse	33
11 Mögliche Erweiterungen	34
11.1 Fachliche Erweiterungen	34
11.2 Programmiertechnische Erweiterungen	35
11.3 Oberflächenanpassungen	35
12 Fazit	35
Anhang	37
A Java-Code	37
A.1 Matrix-Klasse	37
A.2 MatrixException	41
A.3 Controller	42
A.4 View	43
A.5 Main-Klasse	46
Musterantworten	47

1 Einleitung

In diesem Projekt wird ein **Matrizenrechner** in Java entwickelt, der Matrizen verschiedener Größen verarbeiten und grundlegende Rechenoperationen ausführen kann. Ziel ist es, eine Anwendung zu erstellen, die sowohl die mathematischen Grundlagen der Matrizenrechnung als auch die Prinzipien der objektorientierten Programmierung praktisch verbindet.

Der Matrizenrechner soll mehrere zentrale Funktionen bereitstellen. Dazu gehören die **Addition**, **Subtraktion** und **Multiplikation** von Matrizen, das **Transponieren** einer Matrix sowie die Berechnung der **Spur**. Darüber hinaus soll das Programm die **Determinante** für Matrizen bis zur Größe **5x5** bestimmen können. Falls eine Matrix invertierbar ist, soll außerdem die **Inverse** berechnet werden.

Ein besonderer Schwerpunkt liegt auf dem **MVC-Prinzip** (Model-View-Controller). Die Anwendung wird so aufgebaut, dass die Rechenlogik, die grafische Benutzeroberfläche und die Steuerung der Benutzerinteraktionen klar voneinander getrennt sind. Dadurch wird der Code übersichtlicher, besser wartbar und leichter erweiterbar.

Für die Benutzeroberfläche wird eine **GUI** entwickelt, über die Matrizen eingegeben und Ergebnisse angezeigt werden können. Auf diese Weise soll das Programm nicht nur funktional, sondern auch benutzerfreundlich sein.

Zusätzlich wird bei der Berechnung der Determinante eine Hilfsmethode verwendet, die es ermöglicht, die **i-te Zeile** und die **j-te Spalte** einer Matrix zu streichen. Diese Methode ist insbesondere für die rekursive Berechnung von Determinanten hilfreich.

Das Projekt verbindet somit mathematische Verfahren mit einer strukturierten Softwarearchitektur und zeigt, wie sich ein komplexeres Problem in einzelne, gut nachvollziehbare Komponenten zerlegen lässt.

2 Ziel des Projekts

Ziel dieses Projekts ist die Entwicklung eines **Matrizenrechners in Java**, der grundlegende und erweiterte Operationen auf Matrizen durchführen kann. Die Anwendung soll so gestaltet werden, dass sie mathematisch korrekt arbeitet und gleichzeitig durch eine grafische Benutzeroberfläche leicht bedienbar ist.

Im Mittelpunkt steht die Umsetzung eines Programms, das Matrizen addieren, subtrahieren, multiplizieren und transponieren kann. Außerdem soll die **Spur** einer Matrix berechnet werden. Ein weiterer wichtiger Bestandteil ist die Bestimmung der **Determinante** für Matrizen bis zur Größe **5x5**. Falls die Matrix invertierbar ist, soll zusätzlich die **Inverse** berechnet werden.

Ein zentrales Ziel des Projekts ist es außerdem, die Anwendung nach dem **MVC-Prinzip** aufzubauen. Dadurch werden Rechenlogik, Benutzeroberfläche und Steuerung voneinander getrennt. Diese Struktur erleichtert nicht nur die Entwicklung, sondern auch spätere Erweiterungen und die Fehlersuche.

Darüber hinaus soll das Projekt zeigen, wie sich mathematische Inhalte mit objektorientierter Programmierung und einer grafischen Oberfläche verbinden lassen. Die Benutzer sollen Matrizen einfach eingeben, berechnen und die Ergebnisse direkt anzeigen lassen können.

Ein besonderer technischer Aspekt ist die Implementierung einer Hilfsmethode zum Streichen der **i-ten Zeile** und **j-ten Spalte**. Diese Methode wird insbesondere bei der rekursiven Berechnung von Determinanten benötigt und ist ein wichtiger Baustein der mathematischen Logik des Programms.

3 Anforderungen

Damit der Matrizenrechner die gewünschten Aufgaben zuverlässig erfüllen kann, muss das Programm eine Reihe von fachlichen und technischen Anforderungen abdecken. Diese Anforderungen beschreiben, welche Funktionen die Anwendung bereitstellen soll und welche Eigenschaften sie für eine saubere Umsetzung benötigt.

3.1 Funktionale Anforderungen

Zu den wichtigsten funktionalen Anforderungen gehört, dass das Programm Matrizen unterschiedlicher Größe verarbeiten kann. Der Rechner soll insbesondere die folgenden Operationen unterstützen:

- Addition von Matrizen
- Subtraktion von Matrizen
- Multiplikation von Matrizen
- Transponieren einer Matrix
- Berechnung der Spur einer Matrix
- Berechnung der Determinante bis zur Größe **5x5**
- Berechnung der Inverse, falls die Matrix invertierbar ist

Darüber hinaus soll das Programm eine Hilfsmethode bereitstellen, mit der die **i-te Zeile** und die **j-te Spalte** einer Matrix entfernt werden können. Diese Funktion wird vor allem für die rekursive Berechnung der Determinante benötigt.

Die grafische Benutzeroberfläche soll es ermöglichen, Matrizen einzugeben, Operationen auszuwählen und Ergebnisse übersichtlich darzustellen. Die Bedienung soll dabei möglichst einfach und verständlich sein.

3.2 Nicht-funktionale Anforderungen

Neben den fachlichen Funktionen gibt es auch Anforderungen an die Qualität der Umsetzung. Das Programm soll nach dem **MVC-Prinzip** aufgebaut sein, damit Rechenlogik, Benutzeroberfläche und Steuerung klar voneinander getrennt sind. Dadurch wird der Code übersichtlicher und besser wartbar.

Außerdem soll die Anwendung so programmiert werden, dass sie nachvollziehbar, erweiterbar und möglichst fehlerarm ist. Die Klassen sollen sinnvoll benannt und die Methoden so strukturiert werden, dass die einzelnen Schritte der Berechnung leicht verständlich bleiben.

Für die Darstellung der Ergebnisse soll die Methode `toString()` so angepasst werden, dass Matrizen in einer gut lesbaren Form ausgegeben werden können.

3.3 Mathematische Anforderungen

Da es sich um einen Matrizenrechner handelt, müssen die mathematischen Regeln der linearen Algebra korrekt umgesetzt werden. Dazu gehört zum Beispiel, dass Matrizen nur dann addiert oder subtrahiert werden können, wenn sie dieselbe Dimension besitzen. Eine Multiplikation ist nur möglich, wenn die Spaltenzahl der ersten Matrix mit der Zeilenzahl der zweiten Matrix übereinstimmt.

Für die Berechnung der Determinante und der Inversen muss das Programm außerdem prüfen, ob die Matrix quadratisch ist. Nur quadratische Matrizen können eine Determinante besitzen, und nur bei einer von null verschiedenen Determinante ist eine Inverse vorhanden.

3.4 Fehlerbehandlung und Exceptions

Ein wichtiger Bestandteil des Projekts ist die Behandlung von Fehlern durch **Exceptions**. Das Programm soll ungültige Eingaben und nicht erlaubte Operationen erkennen und darauf kontrolliert reagieren, anstatt einfach mit einem Fehler abzubrechen.

Typische Fehlerfälle sind zum Beispiel:

- Addition oder Subtraktion von Matrizen mit unterschiedlichen Dimensionen

- Multiplikation von Matrizen mit nicht passenden Größen
- Berechnung der Determinante oder Inversen bei nicht quadratischen Matrizen
- Berechnung der Inversen bei Matrizen mit Determinante 0
- Ungültige oder nicht numerische Eingaben in der grafischen Benutzeroberfläche

Für diese Fälle sollen passende Exceptions verwendet werden, damit der Benutzer eine verständliche Fehlermeldung erhält. Dadurch wird das Programm robuster und sicherer, weil fehlerhafte Eingaben nicht zu einem Absturz führen.

3.5 Zusammenfassung

Die Anforderungen bilden die Grundlage für die spätere Umsetzung des Projekts. Sie legen fest, welche Funktionen der Matrizenrechner besitzen soll und welche mathematischen, technischen und fehlerbezogenen Bedingungen dabei eingehalten werden müssen.

4 MVC-Prinzip

Das Projekt wird nach dem **MVC-Prinzip** aufgebaut. MVC steht für **Model**, **View** und **Controller**. Dieses Architekturprinzip dient dazu, eine Anwendung in drei klar voneinander getrennte Bereiche zu gliedern. Dadurch wird der Code übersichtlicher, besser wartbar und leichter erweiterbar.

4.1 Model

Das **Model** enthält die gesamte fachliche Logik der Anwendung. In diesem Projekt ist das Model für die Arbeit mit Matrizen zuständig. Hier werden die Matrizen gespeichert und alle mathematischen Operationen implementiert. Dazu gehören unter anderem die Addition, Subtraktion, Multiplikation, das Transponieren, die Berechnung der Spur, der Determinante und der Inverse.

Außerdem gehört auch die Fehlerprüfung zum Model. Wenn zum Beispiel zwei Matrizen nicht die passenden Dimensionen für eine Operation besitzen, soll das Model eine passende Exception auslösen. Auf diese Weise bleibt die mathematische Logik unabhängig von der Benutzeroberfläche.

4.2 View

Die **View** ist für die Darstellung der Benutzeroberfläche zuständig. In diesem Projekt wird dafür eine grafische Oberfläche mit Java erstellt. Die View zeigt Eingabefelder, Schaltflächen und Ausgabebereiche an, über die der Benutzer mit dem Programm interagieren kann.

Wichtig ist dabei, dass die View selbst keine Berechnungen durchführt. Ihre Aufgabe besteht nur darin, Daten anzuzeigen und Benutzereingaben entgegenzunehmen. Dadurch bleibt die Oberfläche klar von der Rechenlogik getrennt.

4.3 Controller

Der **Controller** verbindet Model und View miteinander. Er reagiert auf Benutzeraktionen, zum Beispiel wenn eine Schaltfläche gedrückt wird, liest die Eingaben aus der View aus und übergibt sie an das Model. Anschließend nimmt er das Ergebnis entgegen und sorgt dafür, dass es in der View angezeigt wird.

Der Controller übernimmt damit die Steuerung des gesamten Ablaufs. Er ist die Vermittlungsstelle zwischen Benutzeroberfläche und Rechenlogik und sorgt dafür, dass beide Bereiche sauber zusammenarbeiten.

Merkkasten

Merke: Das MVC-Prinzip trennt die Anwendung in drei Bereiche:

- **Model:** enthält die fachliche Logik und die Daten
- **View:** stellt die Benutzeroberfläche dar
- **Controller:** verbindet View und Model und steuert den Ablauf

Dadurch bleibt der Code übersichtlich, wartbar und gut erweiterbar.

4.4 Vorteile des MVC-Prinzips

Die Trennung in Model, View und Controller bringt mehrere Vorteile mit sich:

- Die einzelnen Teile des Programms sind klar voneinander getrennt.
- Der Code ist leichter zu verstehen und zu warten.
- Änderungen an der Oberfläche beeinflussen die Rechenlogik nicht direkt.
- Neue Funktionen können einfacher ergänzt werden.
- Fehler lassen sich besser eingrenzen und beheben.

Gerade bei einem Matrizenrechner ist diese Struktur sehr sinnvoll, weil mathematische Berechnungen, grafische Darstellung und Benutzerinteraktion unterschiedliche Aufgaben erfüllen. Durch MVC bleibt das Programm sauber aufgebaut und gut erweiterbar.

5 Klassenstruktur

Für die Umsetzung des Matrizenrechners wird das Programm in mehrere Klassen aufgeteilt. Diese Aufteilung orientiert sich am **MVC-Prinzip**, damit die Rechenlogik, die Benutzeroberfläche und die Steuerung klar voneinander getrennt sind. Dadurch bleibt der Code übersichtlich und kann später leichter erweitert oder angepasst werden.

5.1 Matrix

Die Klasse **Matrix** bildet das Herzstück des Projekts. Sie gehört zum **Model** und enthält die eigentlichen Daten der Matrix sowie alle mathematischen Operationen. Dazu zählen unter anderem die Addition, Subtraktion, Multiplikation, das Transponieren, die Berechnung der Spur, der Determinante und der Inverse.

Außerdem enthält diese Klasse Methoden zur Ausgabe der Matrix, zum Beispiel über `toString()`. Auch Hilfsmethoden, etwa zum Streichen einer bestimmten Zeile und Spalte für die Determinantenberechnung, werden hier sinnvoll untergebracht.

5.2 MatrixException

Die Klasse **MatrixException** dient zur Behandlung von Fehlern, die bei ungültigen Matrizenoperationen auftreten können. Sie wird verwendet, wenn zum Beispiel Matrizen nicht die passenden Dimensionen für eine Operation besitzen oder wenn eine Inverse nicht berechnet werden kann.

Durch eine eigene Exception-Klasse kann das Programm Fehler gezielt erkennen und passende Meldungen ausgeben. Das verbessert die Verständlichkeit und macht die Anwendung robuster.

5.3 MatrixView

Die Klasse `MatrixView` gehört zur **View**. Sie ist für die grafische Benutzeroberfläche zuständig. Hier werden Eingabefelder, Buttons, Ausgabeflächen und eventuell weitere Bedienelemente erstellt und angeordnet.

Die View zeigt dem Benutzer die Oberfläche an, nimmt Eingaben entgegen und stellt Ergebnisse dar. Sie führt jedoch keine mathematischen Berechnungen selbst aus.

5.4 MatrixController

Die Klasse `MatrixController` gehört zum **Controller**. Sie verbindet die View mit dem Model. Der Controller reagiert auf Benutzeraktionen, liest die Eingaben aus der Oberfläche aus und übergibt sie an die entsprechenden Methoden der Klasse `Matrix`.

Anschließend verarbeitet der Controller das Ergebnis und sorgt dafür, dass es in der View angezeigt wird. Er übernimmt damit die Steuerung des gesamten Ablaufs zwischen Benutzeroberfläche und Rechenlogik.

5.5 Zusammenfassung

Die Aufteilung in mehrere Klassen sorgt dafür, dass jede Klasse eine klar definierte Aufgabe hat. Die Klasse `Matrix` übernimmt die Berechnungen, `MatrixView` die Darstellung und `MatrixController` die Verbindung zwischen beiden. Die Klasse `MatrixException` ergänzt das Projekt um eine saubere Fehlerbehandlung.

Diese Struktur ist besonders gut geeignet, weil sie das Programm übersichtlich hält und die spätere Erweiterung erleichtert.

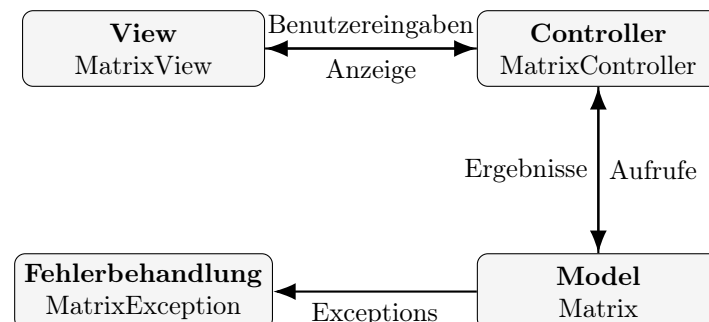


Abbildung 1: Klassenstruktur des Matrizenrechners nach dem MVC-Prinzip

6 Matrix-Klasse

Die Klasse `Matrix` bildet den zentralen Bestandteil des **Models** in der Anwendung. Sie enthält die eigentlichen mathematischen Daten und stellt alle grundlegenden Operationen bereit, die für den Matrizenrechner benötigt werden. Damit übernimmt sie die fachliche Logik des Programms und ist unabhängig von der grafischen Benutzeroberfläche.

Eine Matrix wird in der Klasse als zweidimensionales Feld von Zahlen gespeichert. Zusätzlich müssen die Anzahl der Zeilen und Spalten bekannt sein, damit die Größe der Matrix jederzeit eindeutig bestimmt werden kann. Auf dieser Grundlage können dann alle weiteren Berechnungen durchgeführt werden.

Die Klasse ist so aufgebaut, dass sie sowohl einfache als auch komplexere Operationen unterstützt. Dazu gehören unter anderem das Addieren, Subtrahieren, Multiplizieren und Transponieren von Matrizen. Außerdem werden hier die Berechnung der Spur, der Determinante und der Inverse umgesetzt.

Ein weiterer wichtiger Aspekt ist die Fehlerprüfung. Da nicht jede Operation mit beliebigen Matrizen möglich ist, muss die Klasse bei ungültigen Eingaben passende Exceptions auslösen. So

wird zum Beispiel überprüft, ob zwei Matrizen die gleichen Dimensionen besitzen oder ob eine Matrix überhaupt invertierbar ist.

Die Klasse `Matrix` ist damit der Kern des gesamten Projekts. Sie enthält nicht nur die gespeicherten Werte, sondern auch die komplette Rechenlogik des Matrizenrechners.

6.1 Attribute und Konstruktoren

Die Klasse `Matrix` benötigt einige grundlegende Attribute, um die Daten einer Matrix speichern und verarbeiten zu können. Das wichtigste Attribut ist ein zweidimensionales Zahlenfeld, in dem die einzelnen Matrixwerte abgelegt werden. Zusätzlich werden die Anzahl der Zeilen und Spalten gespeichert, damit die Dimension der Matrix jederzeit bekannt ist.

Typischerweise können die Attribute wie folgt aufgebaut sein:

- `double[][] werte` für die eigentlichen Matrixelemente
- `int zeilen` für die Anzahl der Zeilen
- `int spalten` für die Anzahl der Spalten

Je nach Umsetzung kann es sinnvoll sein, die Werte direkt aus dem zweidimensionalen Array abzuleiten. Dennoch ist es oft hilfreich, Zeilen- und Spaltenanzahl explizit zu speichern, damit die Dimensionen schnell verfügbar sind und die Lesbarkeit des Codes verbessert wird.

6.1.1 Konstruktoren

Die Klasse `Matrix` sollte mindestens einen Konstruktor besitzen, der eine Matrix aus einem vorhandenen zweidimensionalen Array erzeugt. Dabei werden die übergebenen Werte nicht direkt übernommen, sondern in eine eigene interne Kopie übertragen. Dadurch wird verhindert, dass spätere Änderungen am übergebenen Array die Matrix beeinflussen.

Ein möglicher Konstruktor könnte so aussehen:

```
1 public Matrix(double[][] data) {
2     if (data == null || data.length == 0 || data[0].length == 0) {
3         throw new MatrixParseException("Matrix darf nicht leer sein.");
4     }
5
6     int cols = data[0].length;
7
8     for (int i = 1; i < data.length; i++) {
9         if (data[i].length != cols) {
10            throw new MatrixParseException("Ungültige Matrixform.");
11        }
12    }
13
14    this.data = new double[data.length][cols];
15
16    for (int i = 0; i < data.length; i++) {
17        System.arraycopy(data[i], 0, this.data[i], 0, cols);
18    }
19 }
```

Dieser Konstruktor prüft zunächst, ob das übergebene Array gültig und nicht leer ist. Anschließend wird überprüft, ob alle Zeilen die gleiche Länge besitzen, damit nur eine echte rechteckige Matrix akzeptiert wird. Danach wird eine neue interne Matrix angelegt und der Inhalt zeilenweise kopiert.

Zusätzlich kann ein weiterer Konstruktor sinnvoll sein, der eine leere Matrix mit festgelegter Größe erzeugt. Dies ist nützlich, wenn die Matrix zunächst nur angelegt und später mit Werten gefüllt werden soll.

```

1 public Matrix(int zeilen, int spalten) {
2     if (zeilen <= 0 || spalten <= 0) {
3         throw new MatrixParseException("Ungültige Matrixgröße");
4     }
5     data = new double[zeilen][spalten];
6 }

```

Ein solcher Konstruktor erstellt eine Matrix mit der gewünschten Größe und initialisiert alle Einträge automatisch mit dem Wert 0.0.

Merkkasten

Merke: Wenn eine Klasse mehrere Konstruktoren mit unterschiedlicher Parameterliste besitzt, nennt man das **Überladen von Konstruktoren**. So kann ein Objekt auf verschiedene Arten erzeugt werden, je nachdem, welche Informationen bereits vorliegen.

Merkkasten

Merke: Es ist sinnvoll, das übergebene Array nicht direkt zu übernehmen, sondern eine eigene **defensive Kopie** anzulegen. Andernfalls könnte der Aufrufer das ursprüngliche Array nach dem Erzeugen der Matrix weiter verändern und damit auch den Inhalt der Matrix ungewollt beeinflussen. Durch die Kopie bleibt die Matrix **gekapselt** und ihr Zustand von äußeren Änderungen unabhängig.

6.1.2 Bedeutung der Konstruktoren

Die Konstruktoren sorgen dafür, dass eine Matrix in einem gültigen Zustand erzeugt wird. Sie bilden die Grundlage für alle weiteren Berechnungen, da ohne korrekt gespeicherte Werte und Dimensionen keine Operationen möglich wären. Deshalb ist es wichtig, die Konstruktoren klar und robust zu gestalten.

6.2 Methodenübersicht

Die Klasse `Matrix` stellt verschiedene Methoden bereit, mit denen die wichtigsten Operationen auf Matrizen ausgeführt werden können. Diese Methoden bilden die fachliche Rechenlogik des Matrizenrechners und sind so aufgebaut, dass sie klar voneinander getrennte Aufgaben erfüllen.

Zu den grundlegenden Methoden gehören zunächst die Zugriffsmethoden auf die Dimensionen und Matricelemente:

- `zeilen()` zur Ermittlung der Anzahl der Zeilen
- `spalten()` zur Ermittlung der Anzahl der Spalten
- `get(int i, int j)` zum Lesen eines Matricelements
- `set(int i, int j, double value)` zum Setzen eines Matricelements

Diese Methoden ermöglichen einen kontrollierten Zugriff auf die internen Daten der Matrix und unterstützen damit die Kapselung.

Darüber hinaus enthält die Klasse Methoden für die wichtigsten Rechenoperationen:

- `add(Matrix other)` für die Addition zweier Matrizen
- `subtract(Matrix other)` für die Subtraktion zweier Matrizen
- `multiply(Matrix other)` für die Multiplikation zweier Matrizen
- `transpose()` für das Vertauschen von Zeilen und Spalten

- `scalarMultiply(double scalar)` für die Multiplikation einer Matrix mit einem Skalar

Diese Methoden erzeugen jeweils eine neue Matrix als Ergebnis, anstatt die ursprüngliche Matrix direkt zu verändern. Dadurch bleibt der Ausgangszustand des Objekts erhalten.

Für mathematische Eigenschaften der Matrix stehen außerdem folgende Methoden zur Verfügung:

- `trace()` zur Berechnung der Spur
- `determinant()` zur Bestimmung der Determinante
- `inverse()` zur Berechnung der Inversen, sofern die Matrix invertierbar ist

Die Methode `determinant()` arbeitet rekursiv und nutzt dabei die Hilfsmethoden `subMatrix()` und `sign()`. `subMatrix()` erzeugt eine Untermatrix, indem eine bestimmte Zeile und Spalte entfernt werden. `sign()` liefert das Vorzeichen des jeweiligen Kofaktors. Die Methode `adjugate()` berechnet anschließend die Adjunkte der Matrix, die für die Inversenberechnung benötigt wird.

Für die Ausgabe der Matrix wird die Methode `toString()` überschrieben. Sie stellt die Matrix in einer gut lesbaren, zeilenweisen Form dar.

Zusätzlich werden die Methoden `equals()` und `hashCode()` überschrieben. Dadurch können Matrizen sinnvoll miteinander verglichen werden, und Objekte mit gleichem Inhalt liefern denselben Hash-Wert.

Merkkasten

Merke: Die Methoden der Klasse `Matrix` sollten klar voneinander getrennte Aufgaben erfüllen. Zugriffsmethoden lesen oder ändern einzelne Werte, Rechenmethoden berechnen neue Matrizen oder Kennwerte, und Hilfsmethoden unterstützen komplexere Verfahren wie die Berechnung der Determinante oder Inversen.

Stolperfallen

Stolperfalle: Im Model sollten keine Methoden wie `liesMatrixEin()` oder `zeigeMatrix()` stehen. Das Model ist nur für die fachliche Logik und die Berechnungen zuständig. Einlesen und Ausgeben gehören zur **View** beziehungsweise zur **Controller**-Schicht. Wenn diese Aufgaben vermischt werden, wird das Programm unübersichtlich und das MVC-Prinzip verletzt.

6.2.1 Zugriff auf Matricelemente

Damit die Elemente einer Matrix kontrolliert gelesen und verändert werden können, werden die Methoden `get(int i, int j)` und `set(int i, int j, double value)` verwendet. Diese Methoden greifen auf das interne zweidimensionale Array zu, ohne dass von außen direkt auf das Array zugegriffen werden muss.

Die Methode `get(int i, int j)` liefert den Wert an der Position `i, j` zurück. Sie wird verwendet, wenn ein bestimmtes Matricelement gelesen werden soll. Dadurch bleibt der Zugriff auf die Daten gekapselt und es wird vermieden, dass andere Klassen direkt mit dem internen Array arbeiten.

Ein möglicher Aufbau ist:

```
1 public double get(int i, int j) {
2     return werte[i][j];
3 }
```

Die Methode `set(int i, int j, double value)` setzt den Wert an der Position `i, j` auf den übergebenen Wert. Sie wird verwendet, wenn ein Matricelement gezielt verändert werden soll.

```

1 public void set(int i, int j, double value) {
2     werte[i][j] = value;
3 }

```

Durch diese beiden Methoden wird der direkte Zugriff auf das interne Array eingeschränkt. Das verbessert die Kapselung und macht den Code übersichtlicher. Außerdem kann die spätere Fehlerprüfung leichter ergänzt werden, zum Beispiel durch eine Kontrolle, ob die Indizes innerhalb des gültigen Bereichs liegen.

Merkkasten

Merke: Mit `get()` und `set()` wird der Zugriff auf Matrixelemente kontrolliert. So bleibt das interne Array geschützt und die Klasse kann ihre Daten selbst verwalten.

6.2.2 Addition

Die Addition zweier Matrizen ist nur dann möglich, wenn beide Matrizen die gleiche Anzahl an Zeilen und Spalten besitzen. Das bedeutet, dass die beiden Matrizen dieselbe Dimension haben müssen. Andernfalls ist die Operation mathematisch nicht definiert und es muss eine Exception ausgelöst werden.

Bei der Addition werden die entsprechenden Elemente beider Matrizen miteinander addiert. Das Ergebnis ist wieder eine Matrix mit derselben Größe wie die Ausgangsmatrizen. Für jedes Element an der Position i, j gilt also:

$$c_{ij} = a_{ij} + b_{ij}$$

Die Methode zur Addition kann so aufgebaut sein, dass sie eine neue Matrix erzeugt und diese mit den berechneten Werten füllt. Die ursprünglichen Matrizen bleiben dabei unverändert.

Ein möglicher Aufbau der Methode ist:

```

1 public Matrix add(Matrix other) {
2     if (this.zeilen != other.zeilen || this.spalten != other.spalten) {
3         throw new MatrixException("Matrizen muessen die gleiche Groesse
4             haben.");
5     }
6
7     Matrix ergebnis = new Matrix(this.zeilen, this.spalten);
8
9     for (int i = 0; i < zeilen; i++) {
10         for (int j = 0; j < spalten; j++) {
11             ergebnis.set(i, j, this.get(i, j) + other.get(i, j));
12         }
13     }
14
15     return ergebnis;
16 }

```

Diese Implementierung ist übersichtlich, weil sie die Berechnung direkt in einer doppelten Schleife durchführt. Zuerst wird geprüft, ob die Dimensionen passen. Danach wird eine neue Matrix erzeugt und elementweise befüllt.

Stolperfallen

Stolperfalle: Bei der Addition darf nicht vergessen werden, die Dimensionen vorher zu prüfen. Wenn die Matrizen unterschiedlich groß sind, darf keine Berechnung stattfinden, da das Ergebnis sonst mathematisch falsch wäre.

6.2.3 Subtraktion

Die Subtraktion zweier Matrizen ist nur dann möglich, wenn beide Matrizen die gleiche Anzahl an Zeilen und Spalten besitzen. Genau wie bei der Addition müssen die Matrizen also dieselbe Dimension haben. Andernfalls ist die Operation mathematisch nicht definiert und es muss eine Exception ausgelöst werden.

Bei der Subtraktion werden die entsprechenden Elemente beider Matrizen voneinander abgezogen. Das Ergebnis ist wieder eine Matrix mit derselben Größe wie die Ausgangsmatrizen. Für jedes Element an der Position i, j gilt also:

$$c_{ij} = a_{ij} - b_{ij}$$

Die Methode zur Subtraktion kann so aufgebaut sein, dass sie eine neue Matrix erzeugt und diese mit den berechneten Werten füllt. Die ursprünglichen Matrizen bleiben dabei unverändert. Ein möglicher Aufbau der Methode ist:

```
1 public Matrix subtract(Matrix other) {
2     if (this.zeilen != other.zeilen || this.spalten != other.spalten) {
3         throw new MatrixException("Matrizen muessen die gleiche Groesse
4             haben.");
5     }
6     Matrix ergebnis = new Matrix(this.zeilen, this.spalten);
7
8     for (int i = 0; i < zeilen; i++) {
9         for (int j = 0; j < spalten; j++) {
10             ergebnis.set(i, j, this.get(i, j) - other.get(i, j));
11         }
12     }
13
14     return ergebnis;
15 }
```

Auch hier wird zuerst die Dimension geprüft und anschließend elementweise gerechnet. Die Struktur ist fast identisch zur Addition, nur dass statt eines Pluszeichens ein Minuszeichen verwendet wird.

Stolperfallen

Stolperfalle: Die Subtraktion funktioniert nur bei gleich großen Matrizen. Wenn die Dimensionen nicht übereinstimmen, darf keine Berechnung stattfinden, da das Ergebnis sonst mathematisch falsch wäre.

6.2.4 Multiplikation

Die Multiplikation zweier Matrizen ist nur dann möglich, wenn die Anzahl der Spalten der ersten Matrix mit der Anzahl der Zeilen der zweiten Matrix übereinstimmt. Nur dann ist die Operation mathematisch definiert. Ist dies nicht der Fall, muss eine Exception ausgelöst werden.

Hat die erste Matrix die Dimension $n \times m$ und die zweite Matrix die Dimension $m \times k$, dann besitzt das Ergebnis die Dimension $n \times k$. Die Anzahl der Zeilen stammt also von der ersten Matrix, die Anzahl der Spalten von der zweiten Matrix.

Für jedes Element des Ergebnisses wird eine Summe aus Produkten gebildet. Das bedeutet, dass für die Berechnung eines einzelnen Ergebniselements eine zusätzliche Schleife benötigt wird. Für jedes c_{ij} gilt:

$$c_{ij} = \sum_{l=0}^{m-1} a_{il} \cdot b_{lj}$$

Die Methode zur Multiplikation kann daher mit drei verschachtelten Schleifen umgesetzt werden. Die erste Schleife läuft über die Zeilen der ersten Matrix, die zweite über die Spalten der zweiten Matrix und die dritte berechnet die Summe der Produkte.

Ein möglicher Aufbau der Methode ist:

```
1 public Matrix multiply(Matrix other) {
2     if (this.spalten != other.zeilen) {
3         throw new MatrixException("Spaltenzahl der ersten Matrix muss mit
4             der Zeilenzahl der zweiten Matrix uebereinstimmen.");
5     }
6     Matrix ergebnis = new Matrix(this.zeilen, other.spalten);
7
8     for (int i = 0; i < this.zeilen; i++) {
9         for (int j = 0; j < other.spalten; j++) {
10            double summe = 0.0;
11            for (int k = 0; k < this.spalten; k++) {
12                summe += this.get(i, k) * other.get(k, j);
13            }
14            ergebnis.set(i, j, summe);
15        }
16    }
17
18    return ergebnis;
19 }
```

Diese Implementierung ist typisch für die Matrizenmultiplikation, weil für jedes Ergebnisfeld die Produkte der passenden Elemente aufaddiert werden müssen.

Merkkasten

Merke: Bei der Multiplikation von Matrizen gilt: $n \times m$ mal $m \times k$ ergibt $n \times k$. Die gemeinsame Zahl m ist dabei entscheidend, weil die Spaltenzahl der ersten Matrix mit der Zeilenzahl der zweiten Matrix übereinstimmen muss. Außerdem wird für die Berechnung jedes Ergebniselements eine dritte Schleife benötigt, um die Summe der Produkte zu bilden.

Stolperfallen

Stolperfalle: Die Multiplikation ist nur möglich, wenn die Spaltenzahl der ersten Matrix genau der Zeilenzahl der zweiten Matrix entspricht. Diese Bedingung wird oft verwechselt, deshalb sollte sie vor der Berechnung immer geprüft werden.

Hinweis

Hinweis: Eine Division von Matrizen ist nicht direkt definiert. Wenn eine "Division" benötigt wird, arbeitet man stattdessen mit der **Inversen** der Matrix, sofern sie existiert. Das ist jedoch nur möglich, wenn die Matrix quadratisch und invertierbar ist.

6.2.5 Transponieren

Beim Transponieren einer Matrix werden Zeilen und Spalten vertauscht. Aus einem Element an der Position i, j wird also ein Element an der Position j, i . Die transponierte Matrix besitzt dadurch die Dimension $m \times n$, wenn die Ausgangsmatrix die Dimension $n \times m$ hat.

Für jedes Element gilt:

$$c_{ij} = a_{ji}$$

Die Methode zur Transposition erzeugt eine neue Matrix mit vertauschten Dimensionen und trägt die Werte entsprechend um. Die ursprüngliche Matrix bleibt dabei unverändert.

Ein möglicher Aufbau der Methode ist:

```
1 public Matrix transpose() {
2     Matrix ergebnis = new Matrix(this.spalten, this.zeilen);
3
4     for (int i = 0; i < this.zeilen; i++) {
5         for (int j = 0; j < this.spalten; j++) {
6             ergebnis.set(j, i, this.get(i, j));
7         }
8     }
9
10    return ergebnis;
11 }
```

Die Methode ist relativ einfach, weil nur Zeilen und Spalten vertauscht werden müssen. Trotzdem ist sie sehr wichtig, da sie in vielen mathematischen Zusammenhängen verwendet wird.

Merkkasten

Merke: Beim Transponieren werden Zeilen und Spalten vertauscht. Eine Matrix der Größe $n \times m$ wird zu einer Matrix der Größe $m \times n$.

Stolperfallen

Stolperfalle: Beim Transponieren dürfen die Indizes nicht verwechselt werden. Der Wert an $a[i][j]$ muss in der transponierten Matrix an die Position $[j][i]$ geschrieben werden.

6.2.6 Spur

Die Spur einer Matrix ist die Summe aller Elemente auf der Hauptdiagonale. Sie ist nur für extbfquadratische Matrizen definiert, also für Matrizen mit gleich vielen Zeilen und Spalten. Für eine Matrix der Größe $n \times n$ gilt:

$$\text{spur}(\mathbf{A}) = \sum_{i=0}^{n-1} a_{ii}$$

Die Methode zur Berechnung der Spur durchläuft daher nur die Hauptdiagonale der Matrix. Dabei werden die Elemente mit gleichen Zeilen- und Spaltenindizes addiert.

Ein möglicher Aufbau der Methode ist:

```
1 public double trace() {
2     if (this.zeilen != this.spalten) {
3         throw new MatrixException("Die Spur ist nur fuer quadratische
4             Matrizen definiert.");
5     }
6
7     double summe = 0.0;
8     for (int i = 0; i < this.zeilen; i++) {
9         summe += this.get(i, i);
10    }
11    return summe;
12 }
```

Die Methode ist vergleichsweise einfach, da nur die Diagonalelemente betrachtet werden müssen. Trotzdem ist auch hier die Prüfung auf Quadratform wichtig, damit keine mathematisch falsche Berechnung durchgeführt wird.

Merkkasten

Merke: Die Spur ist die Summe der Diagonalelemente einer quadratischen Matrix. Nur Matrizen mit gleicher Zeilen- und Spaltenzahl besitzen eine Spur.

Stolperfallen

Stolperfalle: Die Spur ist nicht für jede Matrix definiert. Vor der Berechnung muss daher immer geprüft werden, ob die Matrix quadratisch ist.

6.2.7 Hilfsmethode zur Erzeugung einer Untermatrix

Für die rekursive Berechnung der Determinante wird eine Hilfsmethode benötigt, mit der aus einer Matrix eine kleinere Untermatrix erzeugt werden kann. Dabei wird eine bestimmte Zeile und eine bestimmte Spalte ausgelassen. Das ist besonders wichtig, wenn die Determinante über die Entwicklung nach der ersten Zeile berechnet wird.

Die Methode erzeugt eine neue Matrix, in der die ausgewählte Zeile und Spalte nicht enthalten sind. Dazu werden zwei verschachtelte Schleifen verwendet. Mit `continue`; kann die jeweils zu streichende Zeile oder Spalte übersprungen werden.

Ein möglicher Aufbau der Methode ist:

```
1  private Matrix subMatrix(int zeile, int spalte) {
2      Matrix result = new Matrix(this.zeilen() - 1, this.spalten() - 1);
3
4      int neueZeile = 0;
5      for (int i = 0; i < this.zeilen(); i++) {
6          if (i == zeile) {
7              continue;
8          }
9
10         int neueSpalte = 0;
11         for (int j = 0; j < this.spalten(); j++) {
12             if (j == spalte) {
13                 continue;
14             }
15
16             result.set(neueZeile, neueSpalte, this.get(i, j));
17             neueSpalte++;
18         }
19         neueZeile++;
20     }
21
22     return result;
23 }
```

Diese Methode ist sehr nützlich, weil sie aus einer Matrix eine kleinere Untermatrix erzeugt. Genau diese Untermatrizen werden für die rekursive Berechnung der Determinante benötigt.

Merkkasten

Merke: Die Hilfsmethode zur Erzeugung einer Untermatrix erstellt eine neue Matrix ohne die ausgewählte Zeile und Spalte. Sie ist vor allem für die rekursive Berechnung der Determinante wichtig, kann aber auch bei anderen Verfahren nützlich sein, bei denen Teilmatrizen benötigt werden.

Stolperfallen

Stolperfalle: Beim Erzeugen der Untermatrix müssen die Zielindizes `neueZeile` und `neueSpalte` korrekt mitgeführt werden. Wenn man hier einen Zähler vergisst, werden die Werte falsch in die neue Matrix geschrieben.

6.2.8 Determinante

Die Determinante ist eine wichtige Zahl, die nur für **quadratische Matrizen** definiert ist. Sie liefert unter anderem Informationen darüber, ob eine Matrix invertierbar ist. Für kleine Matrizen kann die Determinante direkt berechnet werden; für größere Matrizen wird in dieser Implementierung eine rekursive Berechnung verwendet.

Für eine 1×1 -Matrix gilt direkt:

$$\det(\mathbf{A}) = a_{00}$$

Für eine 2×2 -Matrix gilt:

$$\det(\mathbf{A}) = a_{00} \cdot a_{11} - a_{01} \cdot a_{10}$$

Für größere Matrizen erfolgt die Berechnung über die Entwicklung nach der ersten Zeile. Dabei werden kleinere Untermatrizen gebildet, deren Determinanten wiederum berechnet werden. Für diesen Schritt werden die Hilfsmethoden `subMatrix()` und `sign()` verwendet.

Ein möglicher Aufbau der Methode ist:

```
1 public double determinant() {
2     if (this.zeilen() != this.spalten()) {
3         throw new MatrixDimensionException("Die Determinante ist nur fuer
4             quadratische Matrizen definiert.");
5     }
6     if (this.zeilen() == 1) {
7         return this.get(0, 0);
8     }
9     if (this.zeilen() == 2) {
10        return this.get(0, 0) * this.get(1, 1) - this.get(0, 1) * this.get
11            (1, 0);
12    }
13    double summe = 0.0;
14    for (int j = 0; j < this.spalten(); j++) {
15        double vorzeichen = sign(0, j);
16        summe += vorzeichen * this.get(0, j) * subMatrix(0, j).determinant
17            ();
18    }
19    return summe;
20 }
```

Die rekursive Berechnung ist besonders gut geeignet, weil sie das Problem Schritt für Schritt auf kleinere Matrizen zurückführt. Dadurch lässt sich die Determinante auch für größere Matrizen systematisch bestimmen.

Merkkasten

Merke: Die Determinante ist nur für quadratische Matrizen definiert. Für größere Matrizen wird sie in dieser Implementierung rekursiv über die erste Zeile berechnet, indem kleinere Untermatrizen gebildet und deren Determinanten zusammengeführt werden.

Stolperfallen

Stolperfalle: Bei der rekursiven Berechnung muss das Vorzeichen der Kofaktoren beachtet werden. Es wird in der Methode `sign()` nach dem Muster $+, -, +, -, \dots$ bestimmt.

Hinweis

Hinweis: Die rekursive Berechnung der Determinante wächst sehr schnell im Aufwand. Bei einer Entwicklung über Untermatrizen steigt die Anzahl der Teilprobleme stark an; vereinfacht gesprochen liegt das Wachstum im Bereich von $O(n!)$. Deshalb ist dieses Verfahren für größere Matrizen deutlich langsamer als für kleine.

6.2.9 Vorzeichenfunktion

Für die Berechnung der Determinante und der Cofaktoren wird eine Hilfsmethode benötigt, die das Vorzeichen abhängig von der Position eines Elements liefert. Dabei gilt das bekannte Schachbrettmuster der Vorzeichen: an Positionen mit gerader Summe aus Zeilen- und Spaltenindex ist das Vorzeichen positiv, andernfalls negativ.

Ein möglicher Aufbau der Methode ist:

```
1 private double sign(int row, int col) {  
2     return ((row + col) % 2 == 0) ? 1.0 : -1.0;  
3 }
```

Diese Methode ist besonders nützlich, weil sie das Vorzeichenmuster zentral kapselt und dadurch sowohl die Berechnung der Determinante als auch der Adjunkten übersichtlicher macht.

Merkkasten

Merke: Die Vorzeichenfunktion liefert das Vorzeichen eines Kofaktors nach dem Muster $+, -, +, -, \dots$. Dadurch muss dieses Muster nicht an mehreren Stellen im Code erneut implementiert werden.

Stolperfallen

Stolperfalle: Das Vorzeichen hängt nicht nur von der Zeile oder Spalte allein ab, sondern von der Summe beider Indizes.

6.2.10 Adjunkte

Die Adjunkte einer Matrix wird für die Berechnung der Inversen benötigt. Sie entsteht aus der Cofaktormatrix, also aus den Vorzeichen-behafteten Determinanten der jeweiligen Untermatrizen. Anschließend wird diese Cofaktormatrix transponiert.

Ein möglicher Aufbau der Methode ist:

```
1 private Matrix adjugate() {  
2     int n = this.zeilen();  
3     Matrix kofaktoren = new Matrix(n, n);  
4  
5     for (int i = 0; i < n; i++) {  
6         for (int j = 0; j < n; j++) {  
7             Matrix minor = subMatrix(i, j);  
8             double vorzeichen = sign(i, j);  
9             kofaktoren.set(i, j, vorzeichen * minor.determinant());  
10        }  
11    }  
12  
13    return kofaktoren.transpose();  
14 }
```

Die Methode berechnet zunächst für jedes Element den zugehörigen Kofaktor und erzeugt daraus die Cofaktormatrix. Durch das anschließende Transponieren entsteht die Adjunkte, die später bei der Inversenberechnung verwendet wird.

Merkkasten

Merke: Die Adjunkte ist die transponierte Cofaktormatrix. Sie ist ein zentraler Zwischenschritt bei der Berechnung der Inversen.

Stolperfallen

Stolperfalle: Die Adjunkte darf nicht mit der Cofaktormatrix verwechselt werden. Erst nach dem Transponieren liegt die eigentliche Adjunkte vor.

6.2.11 Inverse

Die Inverse einer Matrix ist nur für **quadratische Matrizen** definiert. Außerdem muss die Determinante ungleich null sein, damit eine Inverse überhaupt existiert. Ist die Determinante gleich null, so ist die Matrix nicht invertierbar und es muss eine Exception ausgelöst werden.

Für eine Matrix A gilt bei vorhandener Inversen:

$$A \cdot A^{-1} = I$$

wobei I die Einheitsmatrix ist.

Die Inverse kann über die Adjunkte berechnet werden. Dazu wird die Adjunkte zunächst mit der Methode `adjugate()` bestimmt und anschließend mit dem Kehrwert der Determinante skaliert. Die Skalierung erfolgt über `scalarMultiply()`, sodass keine eigene Schleife für die Division benötigt wird.

Ein möglicher Aufbau der Methode ist:

```
1 public Matrix inverse() {
2     if (zeilen() != spalten()) {
3         throw new MatrixDimensionException("Die Inverse ist nur fuer
4             quadratische Matrizen definiert.");
5     }
6     double det = determinant();
7
8     if (det == 0) {
9         throw new MatrixNotInvertibleException("Die Matrix ist nicht
10             invertierbar, da die Determinante 0 ist.");
11     }
12     if (zeilen() == 1) {
13         Matrix result = new Matrix(1, 1);
14         result.set(0, 0, 1.0 / get(0, 0));
15         return result;
16     }
17
18     return adjugate().scalarMultiply(1.0 / det);
19 }
```

Die Berechnung der Inversen ist deutlich aufwendiger als die anderen Operationen, weil sie auf der Determinante, der Adjunkte und mehreren Hilfsschritten basiert. Deshalb ist es besonders wichtig, die Voraussetzungen vorher genau zu prüfen.

Merkkasten

Merke: Eine Matrix ist nur dann invertierbar, wenn sie quadratisch ist und ihre Determinante ungleich null ist. Eine Matrix mit Determinante 0 heißt **singulär** und besitzt keine Inverse.

Stolperfallen

Stolperfalle: Eine Matrix ist nicht automatisch invertierbar, nur weil sie quadratisch ist. Entscheidend ist, dass die Determinante ungleich null ist.

Hinweis

Hinweis: Für die Berechnung der Inversen werden bereits vorhandene Methoden wiederverwendet. Zuerst wird mit `determinant()` geprüft, ob die Matrix invertierbar ist. Anschließend wird mit `adjugate()` die Adjunkte berechnet und mit `scalarMultiply()` skaliert. Dadurch baut die Inversenberechnung auf bereits getesteten Bausteinen des Programms auf.

6.2.12 toString()

Die Methode `toString()` wird in der Klasse `Matrix` überschrieben, um eine Matrix in einer gut lesbaren Form ausgeben zu können. Standardmäßig liefert `toString()` bei Objekten oft nur eine technische Darstellung, die für den Benutzer kaum hilfreich ist. Deshalb wird die Methode so angepasst, dass die Matrix zeilenweise und übersichtlich dargestellt wird.

Die Ausgabe ist besonders wichtig für die grafische Benutzeroberfläche, da die berechneten Ergebnisse dem Benutzer verständlich angezeigt werden sollen. Jede Zeile der Matrix wird dabei in einer eigenen Zeile ausgegeben, sodass die Struktur der Matrix erkennbar bleibt.

Ein möglicher Aufbau der Methode ist:

```
1  @Override
2  public String toString() {
3      StringBuilder sb = new StringBuilder();
4
5      for (int i = 0; i < this.zeilen; i++) {
6          for (int j = 0; j < this.spalten; j++) {
7              sb.append(String.format("%8.2f", this.get(i, j)));
8          }
9          sb.append("\n");
10     }
11
12     return sb.toString();
13 }
```

In diesem Beispiel wird ein `StringBuilder` verwendet, um die Ausgabe effizient zusammenzusetzen. Mit `String.format()` können die Zahlen formatiert werden, damit die Spalten besser ausgerichtet sind.

Hinweis

Hinweis: Java-Strings sind **unveränderlich** (*immutable*). Das bedeutet, dass bei jeder Verkettung mit `+` ein neuer String erzeugt wird. Für längere Ausgaben ist deshalb `StringBuilder` sinnvoll, weil damit viele Zeichenfolgen effizienter zusammengesetzt werden können.

Merkkasten

Merke: Die Methode `toString()` sollte so überschrieben werden, dass die Matrix für den Benutzer lesbar ausgegeben wird. Das ist besonders wichtig für die Anzeige in der GUI und für Testausgaben.

Stolperfallen

Stolperfalle: Wenn `toString()` nicht angepasst wird, erhält man oft nur eine unübersichtliche Standardausgabe des Objekts. Für Matrizen ist das meist nicht brauchbar.

6.2.13 equals() und hashCode()

Die Methoden `equals()` und `hashCode()` werden in der Klasse `Matrix` überschrieben, um Matrizen sinnvoll miteinander vergleichen zu können. Standardmäßig prüft `equals()` bei Objekten häufig nur, ob zwei Referenzen auf dasselbe Objekt zeigen. Für Matrizen ist jedoch nicht die Speicheradresse entscheidend, sondern der tatsächliche Inhalt.

Daher wird `equals()` so implementiert, dass zwei Matrizen genau dann als gleich gelten, wenn sie dieselben Dimensionen und dieselben Einträge besitzen. Für den Vergleich der zweidimensionalen Datenstruktur wird dabei `Arrays.deepEquals()` verwendet.

Ein möglicher Aufbau der Methoden ist:

```
1  @Override
2  public int hashCode() {
3      final int prime = 31;
4      int result = 1;
5      result = prime * result + Arrays.deepHashCode(data);
6      return result;
7  }
8
9  @Override
10 public boolean equals(Object obj) {
11     if (this == obj)
12         return true;
13     if (obj == null)
14         return false;
15     if (getClass() != obj.getClass())
16         return false;
17     Matrix other = (Matrix) obj;
18     return Arrays.deepEquals(data, other.data);
19 }
```

Die Methode `hashCode()` ergänzt `equals()` und sorgt dafür, dass gleichartige Matrizen auch denselben Hash-Wert erhalten. Das ist besonders wichtig, wenn Matrizen in Datenstrukturen wie `HashSet` oder `HashMap` verwendet werden. In Java gilt dabei die Regel, dass zwei Objekte, die mit `equals()` als gleich erkannt werden, auch denselben `hashCode()` besitzen müssen.

Merkkasten

Merke: `equals()` prüft die inhaltliche Gleichheit zweier Matrizen, während `hashCode()` diese Gleichheit für hashbasierte Datenstrukturen unterstützt. Beide Methoden sollten immer zusammen überschrieben werden.

Stolperfallen

Stolperfalle: Wenn `equals()` überschrieben wird, `hashCode()` aber nicht, kann es zu Fehlern in hashbasierten Datenstrukturen kommen. Beide Methoden müssen daher konsistent implementiert sein.

6.3 Resümee zum Model

Das **Model** bildet den fachlichen Kern des Matrizenrechners. In der Klasse **Matrix** werden die Daten der Matrizen gespeichert und alle mathematischen Operationen umgesetzt. Dazu gehören unter anderem die Addition, Subtraktion, Multiplikation, das Transponieren, die Berechnung der Spur, der Determinante und der Inverse.

Besonders wichtig ist dabei, dass das Model unabhängig von der Benutzeroberfläche arbeitet. Es kümmert sich ausschließlich um die Rechenlogik und die Datenverarbeitung. Ein- und Ausgaben gehören nicht in das Model, sondern werden von der View und dem Controller übernommen. Dadurch bleibt die Aufgabenverteilung im Sinne des MVC-Prinzips klar getrennt.

Durch Hilfsmethoden wie `get()`, `set()` und das Streichen von Zeilen und Spalten wird die Klasse übersichtlich aufgebaut und kann ihre Aufgaben strukturiert erfüllen. Zusätzlich sorgen Exceptions dafür, dass fehlerhafte Eingaben oder unzulässige Operationen sauber behandelt werden.

Damit ist das Model die Grundlage für alle weiteren Bestandteile des Programms und die Voraussetzung dafür, dass die grafische Benutzeroberfläche später mit den Berechnungen arbeiten kann.

7 Exceptions

Bei einem Matrizenrechner können während der Berechnungen verschiedene Fehler auftreten. Diese Fehler sollen nicht zu einem Programmabsturz führen, sondern durch passende Exceptions kontrolliert behandelt werden. Dadurch bleibt die Anwendung robust und der Benutzer erhält verständliche Rückmeldungen, wenn eine Operation nicht ausgeführt werden kann.

Für das Projekt wird eine eigene Exception-Hierarchie verwendet. Die Basisklasse **MatrixException** fasst alle projektspezifischen Fehler zusammen. Von ihr erben die spezialisierteren Exceptions **MatrixParseException**, **MatrixDimensionException** und **MatrixNotInvertibleException**. Diese Aufteilung sorgt dafür, dass Fehler fachlich sinnvoll unterschieden werden können, ohne die gemeinsame Behandlung zu verlieren.

Typische Fehlerfälle sind:

- ungültige oder leere Eingaben beim Erzeugen einer Matrix
- Addition oder Subtraktion von Matrizen mit unterschiedlichen Dimensionen
- Multiplikation von Matrizen mit nicht passenden Größen
- Berechnung der Spur bei nicht quadratischen Matrizen
- Berechnung der Determinante bei nicht quadratischen Matrizen
- Berechnung der Inversen bei Matrizen mit Determinante 0
- ungültige Indizes beim Zugriff auf Matrixelemente

Die Verwendung einer vererbten Exception-Struktur hat den Vorteil, dass im Controller alle Matrixfehler zentral mit `catch (MatrixException ex)` abgefangen werden können. Gleichzeitig bleibt es möglich, die Fehler im Model genauer zu unterscheiden und bei Bedarf gezielt unterschiedliche Meldungen auszugeben. Dadurch wird die Fehlerbehandlung übersichtlich, wiederverwendbar und leicht erweiterbar.

Ein möglicher Aufbau der Exception-Klassen ist:

```
1 public class MatrixException extends RuntimeException {
2     public MatrixException(String nachricht) {
3         super(nachricht);
4     }
5 }
6
7 public class MatrixParseException extends MatrixException {
8     public MatrixParseException(String message) {
9         super(message);
10    }
11 }
12
13 public class MatrixNotInvertibleException extends MatrixException {
14     public MatrixNotInvertibleException(String message) {
15         super(message);
16    }
17 }
18
19 public class MatrixDimensionException extends MatrixException {
20     public MatrixDimensionException(String message) {
21         super(message);
22    }
23 }
```

Alle spezialisierten Exceptions erweitern damit die gemeinsame Basisklasse `MatrixException`. Sie erben deren Verhalten und können trotzdem für konkrete Fehlerarten verwendet werden. Das ist besonders praktisch, weil im Controller alle diese Fehler mit einer gemeinsamen Behandlung abgefangen werden können, während die eigentliche Ursache im Model präzise bleibt.

Merkkasten

Merke: Eine vererbte Exception-Struktur verbindet zwei Vorteile: Im Model können Fehler fachlich genau unterschieden werden, und im Controller können alle Matrixfehler gemeinsam mit `catch (MatrixException ex)` behandelt werden. So bleibt der Code sauber und gut erweiterbar.

Merkkasten

Merke: Die Basisklasse `MatrixException` dient als gemeinsame Oberklasse für alle projektspezifischen Fehler. Dadurch kann der Controller mit einer einzigen `catch`-Anweisung reagieren, ohne dass die einzelnen Fehlerarten ihre fachliche Bedeutung verlieren.

Stolperfallen

Stolperfalle: Exceptions sollten nicht nur geworfen, sondern auch an der richtigen Stelle abgefangen und dem Benutzer verständlich angezeigt werden. Sonst weiß der Benutzer zwar, dass ein Fehler aufgetreten ist, aber nicht, was er tun soll.

8 Controller

Der **Controller** verbindet im MVC-Prinzip das **Model** mit der **View**. Er übernimmt die Steuerung des Ablaufs und reagiert auf die Aktionen des Benutzers. Wenn der Benutzer zum Beispiel auf einen Button klickt oder eine Eingabe bestätigt, wird diese Aktion vom Controller verarbeitet.

Im Projekt des Matrizenrechners liest der Controller die Eingaben aus der grafischen Benutzeroberfläche aus, wandelt sie in geeignete Datenstrukturen um und übergibt sie an das Model.

Dort werden die eigentlichen Berechnungen durchgeführt. Anschließend nimmt der Controller das Ergebnis entgegen und sorgt dafür, dass es in der View angezeigt wird.

Der Controller hat damit eine vermittelnde Funktion. Er enthält selbst keine mathematische Rechenlogik und stellt auch keine Benutzeroberfläche dar. Stattdessen koordiniert er die Zusammenarbeit zwischen den beiden anderen Schichten.

Zu den Aufgaben des Controllers gehören unter anderem:

- Auslesen der Benutzereingaben aus der View
- Umwandeln der Eingaben in Matrizenobjekte
- Aufruf der Methoden der Klasse **Matrix**
- Behandlung von Exceptions bei fehlerhaften Eingaben oder unzulässigen Operationen
- Weitergabe der Ergebnisse an die View

Besonders wichtig ist dabei, dass der Controller die Fehlerbehandlung sinnvoll organisiert. Wenn das Model eine Exception wirft, kann der Controller diese abfangen und eine verständliche Meldung an die Benutzeroberfläche weitergeben. So bleibt das Programm stabil und der Benutzer erhält direkt Rückmeldung über den Fehler.

Ein Controller ist also die zentrale Schaltstelle zwischen Eingabe, Berechnung und Ausgabe. Ohne ihn würden View und Model nicht sauber zusammenarbeiten.

Merkkasten

Merke: Der Controller steuert den Ablauf zwischen View und Model. Er liest Eingaben ein, ruft Berechnungen im Model auf und gibt die Ergebnisse an die View zurück.

Stolperfallen

Stolperfalle: Der Controller sollte keine Rechenlogik enthalten. Seine Aufgabe ist es, die Abläufe zu koordinieren, nicht selbst mathematische Operationen durchzuführen.

8.1 Controller-Umsetzung in Java

Der Controller verbindet im MVC-Prinzip die grafische Benutzeroberfläche mit der Rechenlogik des Models. Er reagiert auf Benutzeraktionen, liest die Eingaben aus der View aus und ruft die passenden Methoden der Klasse **Matrix** auf. Das Ergebnis wird anschließend wieder an die View zurückgegeben.

Für die Umsetzung in Java werden häufig **ActionListener** verwendet, da sie auf Button-Klicks reagieren können. Jeder Button der Benutzeroberfläche erhält dabei einen eigenen Listener, der die jeweilige Operation ausführt.

Ein möglicher Aufbau des Controllers ist:

```
1  import java.awt.event.ActionEvent;
2  import java.awt.event.ActionListener;
3
4  public class MatrixController {
5
6      private final MatrixView view;
7
8      public MatrixController(MatrixView view) {
9          this.view = view;
10         this.view.setAddActionListener(new AddActionListener());
11         this.view.setSubtractActionListener(new SubtractActionListener());
12         this.view.setMultiplyActionListener(new MultiplyActionListener());
13         this.view.setTransposeActionListener(new TransposeActionListener()
            );
14     }
15 }
```



```

14     this.view.setTraceActionListener(new TraceActionListener());
15     this.view.setDeterminantActionListener(new
16         DeterminantActionListener());
17     this.view.setInverseActionListener(new InverseActionListener());
18 }
19 private class AddActionListener implements ActionListener {
20     @Override
21     public void actionPerformed(ActionEvent e) {
22         try {
23             Matrix a = view.getMatrixA();
24             Matrix b = view.getMatrixB();
25             Matrix result = a.add(b);
26             view.showResult(result.toString());
27         } catch (MatrixException ex) {
28             view.showError(ex.getMessage());
29         }
30     }
31 }
32
33 private class SubtractActionListener implements ActionListener {
34     @Override
35     public void actionPerformed(ActionEvent e) {
36         try {
37             Matrix a = view.getMatrixA();
38             Matrix b = view.getMatrixB();
39             Matrix result = a.subtract(b);
40             view.showResult(result.toString());
41         } catch (MatrixException ex) {
42             view.showError(ex.getMessage());
43         }
44     }
45 }
46
47 private class MultiplyActionListener implements ActionListener {
48     @Override
49     public void actionPerformed(ActionEvent e) {
50         try {
51             Matrix a = view.getMatrixA();
52             Matrix b = view.getMatrixB();
53             Matrix result = a.multiply(b);
54             view.showResult(result.toString());
55         } catch (MatrixException ex) {
56             view.showError(ex.getMessage());
57         }
58     }
59 }
60
61 private class TransposeActionListener implements ActionListener {
62     @Override
63     public void actionPerformed(ActionEvent e) {
64         try {
65             Matrix a = view.getMatrixA();
66             Matrix result = a.transpose();
67             view.showResult(result.toString());
68         } catch (MatrixException ex) {
69             view.showError(ex.getMessage());
70         }
71     }
72 }
73

```

```

74     private class TraceActionListener implements ActionListener {
75         @Override
76         public void actionPerformed(ActionEvent e) {
77             try {
78                 Matrix a = view.getMatrixA();
79                 double result = a.trace();
80                 view.showResult(String.valueOf(result));
81             } catch (MatrixException ex) {
82                 view.showError(ex.getMessage());
83             }
84         }
85     }
86
87     private class DeterminantActionListener implements ActionListener {
88         @Override
89         public void actionPerformed(ActionEvent e) {
90             try {
91                 Matrix a = view.getMatrixA();
92                 double result = a.determinant();
93                 view.showResult(String.valueOf(result));
94             } catch (MatrixException ex) {
95                 view.showError(ex.getMessage());
96             }
97         }
98     }
99
100    private class InverseActionListener implements ActionListener {
101        @Override
102        public void actionPerformed(ActionEvent e) {
103            try {
104                Matrix a = view.getMatrixA();
105                Matrix result = a.inverse();
106                view.showResult(result.toString());
107            } catch (MatrixException ex) {
108                view.showError(ex.getMessage());
109            }
110        }
111    }
112 }

```

Der Controller übernimmt damit die Steuerung zwischen Benutzeroberfläche und Rechenlogik. Er selbst führt keine mathematischen Berechnungen durch, sondern leitet die Eingaben an das Model weiter und verarbeitet die Ergebnisse.

Hinweis

Hinweis: Statt eigener Listener-Klassen können auch **Lambda-Ausdrücke** verwendet werden, da `ActionListener` ein funktionales Interface ist. Das macht den Code kürzer und oft übersichtlicher, wenn die Aktion nur aus wenigen Zeilen besteht.

Merkkasten

Merke: Der Controller ist die Vermittlungsstelle zwischen View und Model. Er reagiert auf Benutzeraktionen, ruft die passenden Methoden im Model auf und gibt die Ergebnisse an die View zurück.

Stolperfallen

Stolperfalle: Der Controller sollte keine Rechenlogik enthalten. Seine Aufgabe ist es, Eingaben zu verarbeiten und Methoden aufzurufen, nicht selbst mathematische Operationen durchzuführen.

8.2 Controller-Umsetzung mit Lambda-Ausdrücken

Da `ActionListener` ein funktionales Interface ist, können die Ereignisbehandlungen im Controller auch mit **Lambda-Ausdrücken** umgesetzt werden. Das macht den Code kompakter und reduziert die Anzahl zusätzlicher Klassen.

Ein möglicher Aufbau ist:

```
1  import java.awt.event.ActionEvent;
2  import java.awt.event.ActionListener;
3
4  public class MatrixController {
5
6      private final MatrixView view;
7
8      public MatrixController(MatrixView view) {
9          this.view = view;
10
11          this.view.setAddActionListener(e -> handleAdd());
12          this.view.setSubtractActionListener(e -> handleSubtract());
13          this.view.setMultiplyActionListener(e -> handleMultiply());
14          this.view.setTransposeActionListener(e -> handleTranspose());
15          this.view.setTraceActionListener(e -> handleTrace());
16          this.view.setDeterminantActionListener(e -> handleDeterminant());
17          this.view.setInverseActionListener(e -> handleInverse());
18      }
19
20      private void handleAdd() {
21          try {
22              Matrix a = view.getMatrixA();
23              Matrix b = view.getMatrixB();
24              Matrix result = a.add(b);
25              view.showResult(result.toString());
26          } catch (MatrixException ex) {
27              view.showError(ex.getMessage());
28          }
29      }
30
31      private void handleSubtract() {
32          try {
33              Matrix a = view.getMatrixA();
34              Matrix b = view.getMatrixB();
35              Matrix result = a.subtract(b);
36              view.showResult(result.toString());
37          } catch (MatrixException ex) {
38              view.showError(ex.getMessage());
39          }
40      }
41
42      private void handleMultiply() {
43          try {
44              Matrix a = view.getMatrixA();
45              Matrix b = view.getMatrixB();
46              Matrix result = a.multiply(b);
47              view.showResult(result.toString());
```

```

48     } catch (MatrixException ex) {
49         view.showError(ex.getMessage());
50     }
51 }
52
53 private void handleTranspose() {
54     try {
55         Matrix a = view.getMatrixA();
56         Matrix result = a.transpose();
57         view.showResult(result.toString());
58     } catch (MatrixException ex) {
59         view.showError(ex.getMessage());
60     }
61 }
62
63 private void handleTrace() {
64     try {
65         Matrix a = view.getMatrixA();
66         double result = a.trace();
67         view.showResult(String.valueOf(result));
68     } catch (MatrixException ex) {
69         view.showError(ex.getMessage());
70     }
71 }
72
73 private void handleDeterminant() {
74     try {
75         Matrix a = view.getMatrixA();
76         double result = a.determinant();
77         view.showResult(String.valueOf(result));
78     } catch (MatrixException ex) {
79         view.showError(ex.getMessage());
80     }
81 }
82
83 private void handleInverse() {
84     try {
85         Matrix a = view.getMatrixA();
86         Matrix result = a.inverse();
87         view.showResult(result.toString());
88     } catch (MatrixException ex) {
89         view.showError(ex.getMessage());
90     }
91 }
92 }

```

Die Lambda-Version ist besonders praktisch, weil die Registrierung der Ereignisbehandlung direkt im Konstruktor erfolgt. Dadurch ist sofort erkennbar, welcher Button welche Aktion auslöst.

Hinweis

Hinweis: Lambda-Ausdrücke können hier verwendet werden, weil `ActionListener` nur eine abstrakte Methode besitzt. Solche Interfaces nennt man **funktionale Interfaces**. Sie eignen sich besonders gut für kurze Ereignisbehandlungen.

Merkkasten

Merke: Lambda-Ausdrücke machen den Controller oft kompakter und lesbarer. Besonders bei einfachen Button-Aktionen ist diese Schreibweise meist übersichtlicher als eigene Listener-Klassen.

Stolperfallen

Stolperfalle: Lambda-Ausdrücke sind nur möglich, wenn das verwendete Interface genau eine abstrakte Methode besitzt. Bei mehreren abstrakten Methoden funktioniert diese Schreibweise nicht.

8.3 Resümee zum Controller

Der **Controller** übernimmt im MVC-Prinzip die Vermittlerrolle zwischen **View** und **Model**. Er reagiert auf Benutzeraktionen, liest die Eingaben aus der grafischen Oberfläche aus und ruft die passenden Methoden der Klasse **Matrix** auf. Die Ergebnisse werden anschließend wieder an die View zurückgegeben.

Besonders wichtig ist dabei, dass der Controller selbst keine mathematischen Berechnungen durchführt. Seine Aufgabe besteht darin, den Ablauf zu steuern und die einzelnen Schichten sauber miteinander zu verbinden. Dadurch bleibt die Rechenlogik im Model und die Darstellung in der View getrennt.

Für die Umsetzung in Java können entweder eigene **ActionListener**-Klassen oder **Lambda-Ausdrücke** verwendet werden. Beide Varianten eignen sich, um Button-Klicks zu verarbeiten. Die Lambda-Variante ist dabei oft kürzer und übersichtlicher.

Damit ist der Controller ein zentrales Bindeglied der Anwendung und sorgt dafür, dass der Matrizenrechner auf Benutzereingaben korrekt reagieren kann.

9 View

Die **View** ist im MVC-Prinzip für die Darstellung der Benutzeroberfläche zuständig. Sie zeigt dem Benutzer die Eingabefelder, Schaltflächen und Ausgabebereiche an und ermöglicht damit die Interaktion mit dem Programm. Im Matrizenrechner ist die View also der Teil, den der Benutzer direkt sieht und bedient.

Wichtig ist, dass die View selbst keine mathematischen Berechnungen durchführt. Ihre Aufgabe besteht ausschließlich darin, Daten anzuzeigen und Benutzereingaben entgegenzunehmen. Die eigentliche Rechenlogik liegt im Model, während der Controller die Verbindung zwischen beiden herstellt.

Für die konkrete Umsetzung der View bedeutet das, dass zunächst die grafische Oberfläche aufgebaut wird. Dazu gehören die Eingabefelder für die beiden Matrizen, die Schaltflächen für die verschiedenen Operationen sowie ein Ausgabebereich für das Ergebnis. Die Oberfläche ist so gestaltet, dass Eingabe, Aktion und Ausgabe klar voneinander getrennt sind.

Zusätzlich übernimmt die View das Auslesen und Umwandeln der Benutzereingaben. Die Methode `parseMatrix()` liest den Text aus den Eingabefeldern, bestimmt daraus die Anzahl der Zeilen und Spalten und wandelt die Werte in ein zweidimensionales Array um. Anschließend wird daraus ein Matrixobjekt erzeugt. Dadurch kann der Benutzer Matrizen in einer einfachen Textform eingeben.

Außerdem stellt die View Methoden bereit, mit denen der Controller die Buttons mit Aktionen verknüpfen kann. Über die `set...ActionListener()`-Methoden werden die Schaltflächen mit den passenden Ereignisbehandlungen verbunden. So bleibt die View selbst frei von fachlicher Logik und dient nur als Oberfläche und Schnittstelle zur Benutzerinteraktion.

Die View sollte so gestaltet sein, dass sie übersichtlich und leicht verständlich ist. Der Benutzer muss schnell erkennen können, wo Matrizen eingegeben werden, welche Operationen möglich sind und wo das Ergebnis erscheint. Eine klare Struktur der Oberfläche ist deshalb besonders wichtig.

Außerdem sollte die View möglichst wenig eigene Logik enthalten. Sie stellt lediglich die Oberfläche bereit und übergibt Benutzeraktionen an den Controller. Dadurch bleibt die Aufgabenverteilung im Sinne des MVC-Prinzips sauber getrennt.

Merkkasten

Merke: Die View ist nur für die Darstellung und die Benutzereingaben zuständig. Berechnungen gehören nicht in die View, sondern in das Model.

Stolperfallen

Stolperfalle: Die View sollte nicht mit Rechenlogik überladen werden. Wenn Berechnungen direkt in der Oberfläche stattfinden, wird das MVC-Prinzip verletzt und der Code unübersichtlich.

9.1 Bestandteile der Oberfläche

Die grafische Benutzeroberfläche des Matrizenrechners besteht aus mehreren klar getrennten Bereichen. Diese Bereiche sorgen dafür, dass der Benutzer Matrizen eingeben, Operationen auswählen und Ergebnisse übersichtlich ablesen kann.

Zu den wichtigsten Bestandteilen gehören zunächst die Eingabefelder für die Matrizen. In der konkreten Umsetzung werden dafür zwei Textbereiche verwendet, in die der Benutzer die Werte zeilenweise eingeben kann. Die Eingaben werden später von der View verarbeitet und in Matrixobjekte umgewandelt.

Ein weiterer zentraler Bestandteil sind die Schaltflächen für die verschiedenen Operationen. Über diese Buttons kann der Benutzer zum Beispiel die Addition, Subtraktion, Multiplikation, das Transponieren, die Berechnung der Spur, der Determinante oder der Inversen auslösen.

Zusätzlich benötigt die Oberfläche einen Ausgabebereich, in dem das Ergebnis angezeigt wird. Dort soll die berechnete Matrix oder ein numerischer Wert, etwa bei der Spur oder der Determinante, gut lesbar dargestellt werden. Auch Fehlermeldungen können über die Oberfläche ausgegeben werden.

Die einzelnen Bestandteile sollten so angeordnet sein, dass die Bedienung logisch und intuitiv bleibt. Der Benutzer sollte ohne lange Suche erkennen können, wo er Eingaben macht, welche Operationen verfügbar sind und wo das Ergebnis erscheint.

Merkkasten

Merke: Eine gute Oberfläche ist übersichtlich aufgebaut und trennt Eingabe, Aktion und Ausgabe klar voneinander.

Stolperfallen

Stolperfalle: Wenn zu viele Elemente unstrukturiert auf der Oberfläche verteilt sind, wird die Bedienung schnell unübersichtlich. Deshalb sollten Eingabefelder, Buttons und Ausgaben klar gruppiert werden.

9.2 Zusammenarbeit mit Controller und Model

Die View arbeitet im MVC-Prinzip eng mit dem Controller zusammen, während das Model für die eigentlichen Berechnungen zuständig ist. Diese Zusammenarbeit sorgt dafür, dass die Aufgaben im Programm klar verteilt bleiben und jede Schicht nur ihre eigene Aufgabe übernimmt.

Wenn der Benutzer in der View eine Eingabe macht oder auf einen Button klickt, wird diese Aktion nicht direkt von der View verarbeitet, sondern über die im Controller registrierten ActionListener weitergegeben. Der Controller liest die Eingaben aus der View aus, wandelt sie in geeignete Matrixobjekte um und ruft anschließend die passende Methode im Model auf. Das Model führt dann die mathematische Berechnung durch und liefert das Ergebnis zurück.

Die View selbst kennt die Rechenlogik nicht. Sie stellt lediglich die Oberfläche bereit, verarbeitet die Eingaben in eine für das Programm nutzbare Form und zeigt Ergebnisse oder Fehlermeldungen an, die ihr vom Controller übergeben werden. Dadurch bleibt die Benutzeroberfläche unabhängig von den mathematischen Details.

Diese klare Trennung hat mehrere Vorteile: Die Oberfläche kann verändert werden, ohne die Rechenlogik anzupassen, und die Berechnungen können getestet werden, ohne die GUI zu benötigen. Außerdem wird der Code insgesamt übersichtlicher und leichter wartbar.

Merkkasten

Merke: Die View nimmt Eingaben entgegen und zeigt Ergebnisse an, der Controller vermittelt zwischen View und Model, und das Model führt die eigentlichen Berechnungen aus.

Stolperfallen

Stolperfalle: Die View sollte nicht direkt auf das Model zugreifen. Alle Verbindungen zwischen Oberfläche und Rechenlogik laufen über den Controller, sonst wird das MVC-Prinzip verletzt.

9.3 View-Umsetzung in Java

Die View wird im Projekt mit **Java Swing** umgesetzt. Swing eignet sich gut für eine grafische Oberfläche, weil damit Fenster, Eingabefelder, Buttons und Ausgabebereiche einfach erstellt werden können. Die View stellt dabei nur die Oberfläche bereit und enthält keine mathematische Rechenlogik.

In der konkreten Umsetzung besteht die View aus zwei Eingabebereichen für die Matrizen, mehreren Schaltflächen für die Rechenoperationen und einem Ausgabebereich für das Ergebnis. Die Eingaben werden als Text erfasst, in der Methode `parseMatrix()` in ein zweidimensionales Array umgewandelt und anschließend in ein Matrixobjekt überführt. Dabei werden die Zeilen über Zeilenumbrüche und die Werte innerhalb einer Zeile über Leerzeichen getrennt; Dezimalzahlen können auch mit Komma eingegeben werden.

Zusätzlich stellt die View Methoden bereit, mit denen der Controller die Buttons mit Aktionen verknüpfen kann. Über die `set...ActionListener()`-Methoden werden die Schaltflächen mit den passenden Ereignisbehandlungen verbunden. Außerdem kann die View über `showResult()` das Ergebnis anzeigen und über `showError()` Fehlermeldungen ausgeben.

Ein möglicher Aufbau der View ist:

```
1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.ActionListener;
4
5  public class MatrixView extends JFrame {
6
7      private final JTextArea matrixATextArea;
8      private final JTextArea matrixBTextArea;
9      private final JTextArea resultTextArea;
10
11     private final JButton addButton;
12     private final JButton subtractButton;
13     private final JButton multiplyButton;
14     private final JButton transposeButton;
15     private final JButton traceButton;
16     private final JButton determinantButton;
17     private final JButton inverseButton;
18
19     public MatrixView() {
20         setTitle("Matrizenrechner");
21         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```

22     setSize(900, 600);
23     setLocationRelativeTo(null);
24
25     matrixATextArea = new JTextArea(8, 20);
26     matrixBTextArea = new JTextArea(8, 20);
27     resultTextArea = new JTextArea(10, 30);
28     resultTextArea.setEditable(false);
29
30     addButton = new JButton("+");
31     subtractButton = new JButton("-");
32     multiplyButton = new JButton("*");
33     transposeButton = new JButton("T");
34     traceButton = new JButton("Spur");
35     determinantButton = new JButton("Determinante");
36     inverseButton = new JButton("Inverse");
37
38     JPanel inputPanel = new JPanel(new GridLayout(1, 2, 10, 10));
39     inputPanel.add(new JScrollPane(matrixATextArea));
40     inputPanel.add(new JScrollPane(matrixBTextArea));
41
42     JPanel buttonPanel = new JPanel(new GridLayout(2, 4, 10, 10));
43     buttonPanel.add(addButton);
44     buttonPanel.add(subtractButton);
45     buttonPanel.add(multiplyButton);
46     buttonPanel.add(transposeButton);
47     buttonPanel.add(traceButton);
48     buttonPanel.add(determinantButton);
49     buttonPanel.add(inverseButton);
50
51     JPanel resultPanel = new JPanel(new BorderLayout());
52     resultPanel.add(new JLabel("Ergebnis:"), BorderLayout.NORTH);
53     resultPanel.add(new JScrollPane(resultTextArea), BorderLayout.
54         CENTER);
55
56     setLayout(new BorderLayout(10, 10));
57     add(inputPanel, BorderLayout.NORTH);
58     add(buttonPanel, BorderLayout.CENTER);
59     add(resultPanel, BorderLayout.SOUTH);
60
61     public Matrix getMatrixA() {
62         return parseMatrix(matrixATextArea.getText());
63     }
64
65     public Matrix getMatrixB() {
66         return parseMatrix(matrixBTextArea.getText());
67     }
68
69     public void showResult(String text) {
70         resultTextArea.setText(text);
71     }
72
73     public void showError(String message) {
74         JOptionPane.showMessageDialog(this, message, "Fehler", JOptionPane.
75             ERROR_MESSAGE);
76     }
77
78     public void setAddActionListener(ActionListener listener) {
79         addButton.addActionListener(listener);
80     }

```



```

81     public void setSubtractActionListener(ActionListener listener) {
82         subtractButton.addActionListener(listener);
83     }
84
85     public void setMultiplyActionListener(ActionListener listener) {
86         multiplyButton.addActionListener(listener);
87     }
88
89     public void setTransposeActionListener(ActionListener listener) {
90         transposeButton.addActionListener(listener);
91     }
92
93     public void setTraceActionListener(ActionListener listener) {
94         traceButton.addActionListener(listener);
95     }
96
97     public void setDeterminantActionListener(ActionListener listener) {
98         determinantButton.addActionListener(listener);
99     }
100
101     public void setInverseActionListener(ActionListener listener) {
102         inverseButton.addActionListener(listener);
103     }
104
105     private Matrix parseMatrix(String input) {
106         String[] rows = input.trim().split("\\n");
107         int zeilen = rows.length;
108         String[] firstRow = rows[0].trim().split("\\s+");
109         int spalten = firstRow.length;
110
111         double[][] werte = new double[zeilen][spalten];
112
113         for (int i = 0; i < zeilen; i++) {
114             String[] values = rows[i].trim().split("\\s+");
115             for (int j = 0; j < spalten; j++) {
116                 werte[i][j] = Double.parseDouble(values[j].replace(",", "."));
117             }
118         }
119
120         return new Matrix(werte);
121     }
122 }

```

Diese View stellt die Eingabefelder, Buttons und den Ausgabebereich bereit. Über die Methoden `setAddActionListener()` und die weiteren Listener-Methoden kann der Controller die Buttons mit Aktionen verbinden. Die Methoden `getMatrixA()` und `getMatrixB()` lesen die Eingaben aus den Textfeldern aus und wandeln sie in Matrixobjekte um.

Die Methode `parseMatrix()` verarbeitet die Benutzereingabe und erzeugt daraus ein zweidimensionales Array, das anschließend an den Konstruktor der Klasse `Matrix` übergeben wird. Dabei werden die Zeilen über Zeilenumbrüche und die einzelnen Werte innerhalb einer Zeile über Leerzeichen getrennt. Dezimalzahlen können nun auch mit Komma eingegeben werden, da das Komma vor der Umwandlung in `double` durch einen Punkt ersetzt wird.

Hinweis

Hinweis: Die Methode `parseMatrix()` ist hier Teil der View, weil sie die Benutzereingabe in eine Matrix umwandelt. Wenn Sie die Aufgaben noch strenger trennen möchten, kann diese Umwandlung auch in den Controller ausgelagert werden.

Merkkasten

Merke: Die View stellt nur die Oberfläche bereit. Sie zeigt Eingaben, Buttons und Ergebnisse an, während der Controller die Aktionen verarbeitet und das Model die Berechnungen durchführt. Durch die Unterstützung von Komma als Dezimaltrennzeichen wird die Eingabe für den Benutzer zusätzlich vereinfacht.

Merkkasten

Merke: Die einzelnen Elemente der View haben unterschiedliche Aufgaben: Eingabefelder nehmen Matrizen auf, Buttons lösen Aktionen aus und der Ausgabebereich zeigt Ergebnisse oder Fehlermeldungen an. Ein Button wie + führt dabei nicht selbst die Addition aus, sondern gibt den Befehl an den Controller weiter.

Stolperfallen

Stolperfalle: Die View sollte nicht mit Rechenlogik überladen werden. Wenn zu viele Berechnungen oder Prüfungen direkt in der Oberfläche stattfinden, wird das MVC-Prinzip unnötig vermischt.

Hinweis

Hinweis: Ein Element der View wird typischerweise in mehreren Schritten aufgebaut: Zuerst wird es als Attribut deklariert, zum Beispiel mit `private final JButton determinantButton;`. Danach wird es im Konstruktor initialisiert, etwa mit `determinantButton = new JButton("Determinante");`. Anschließend wird es einem passenden Panel hinzugefügt, zum Beispiel mit `buttonPanel.add(determinantButton);`. Damit der Button eine Aktion auslösen kann, stellt die View außerdem eine Methode wie `setDeterminantActionListener(...)` bereit. Die eigentliche Rechenlogik gehört dabei nicht in die View, sondern in den Controller und das Model. Eigenschaften wie Größe, Schrift oder Farbe können zusätzlich über die Gestaltung der Oberfläche festgelegt werden.

9.4 Resümee zur View

Die **View** ist der Teil des Matrizenrechners, den der Benutzer direkt sieht und bedient. Sie stellt die Eingabefelder, Schaltflächen und Ausgabebereiche bereit und sorgt damit für die grafische Darstellung der Anwendung. Ihre Aufgabe besteht darin, Eingaben entgegenzunehmen und Ergebnisse oder Fehlermeldungen anzuzeigen.

Wichtig ist, dass die View selbst keine mathematischen Berechnungen durchführt. Diese gehören ausschließlich in das Model. Die View arbeitet stattdessen mit dem Controller zusammen, der die Benutzereingaben verarbeitet und die passenden Methoden im Model aufruft.

Durch diese klare Trennung bleibt die Benutzeroberfläche übersichtlich und unabhängig von der Rechenlogik. Das macht das Programm besser wartbar und leichter erweiterbar. Außerdem kann die Oberfläche später verändert werden, ohne dass die mathematischen Funktionen angepasst werden müssen.

Damit bildet die View den sichtbaren und interaktiven Teil des MVC-Prinzips und ist die Schnittstelle zwischen Benutzer und Anwendung.

10 Main-Klasse

Die **Main-Klasse** bildet den Einstiegspunkt des Programms. In ihr wird die Anwendung gestartet, indem die benötigten Objekte für **View**, **Model** und **Controller** erzeugt und miteinander verbunden werden. Sie sorgt also dafür, dass der Matrizenrechner als vollständige Anwendung ausgeführt werden kann.

Typischerweise enthält die Main-Klasse nur wenig Logik. Ihre Hauptaufgabe besteht darin, die einzelnen Bestandteile des Programms zu initialisieren und die grafische Benutzeroberfläche sichtbar zu machen. Dadurch bleibt die Startklasse übersichtlich und klar strukturiert.

Ein möglicher Aufbau ist:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         MatrixView view = new MatrixView();  
4         MatrixController controller = new MatrixController(view);  
5         view.setVisible(true);  
6     }  
7 }
```

In diesem Beispiel wird zuerst die View erzeugt. Danach wird der Controller mit der View verbunden, damit die Benutzeraktionen verarbeitet werden können. Anschließend wird die Oberfläche sichtbar gemacht.

Diese Trennung ist wichtig für die **Wartbarkeit** des Programms. Wenn der Programmstart klar in einer eigenen Klasse organisiert ist, lassen sich spätere Änderungen leichter vornehmen. Auch die **Erweiterbarkeit** wird verbessert, weil neue Bestandteile des Programms ergänzt werden können, ohne die gesamte Startlogik zu verändern.

Merkkasten

Merke: Die Main-Klasse ist nur für den Start der Anwendung zuständig. Sie verbindet View und Controller und sollte selbst keine fachliche Logik enthalten. Eine klare Startstruktur verbessert die **Wartbarkeit** und **Erweiterbarkeit** des Programms.

Stolperfallen

Stolperfalle: Die Main-Klasse sollte nicht mit Rechenlogik oder GUI-Details überladen werden. Ihre Aufgabe ist nur der saubere Programmstart und die Verbindung der Hauptkomponenten.

11 Mögliche Erweiterungen

11.1 Fachliche Erweiterungen

Das Projekt des Matrizenrechners kann fachlich um weitere mathematische Funktionen ergänzt werden. Eine naheliegende Erweiterung ist die Berechnung von Eigenwerten und Eigenvektoren, da diese zu den zentralen Konzepten der linearen Algebra gehören. Auch die Bestimmung des Rangs einer Matrix oder das Lösen linearer Gleichungssysteme wären sinnvolle Ergänzungen.

Darüber hinaus könnten weitere Operationen wie das Potenzieren von Matrizen oder die Berechnung spezieller Matrixformen implementiert werden. Solche Erweiterungen würden den Matrizenrechner inhaltlich deutlich aufwerten und ihn für mathematische Anwendungen vielseitiger machen.

Merkkasten

Merke: Fachliche Erweiterungen betreffen die mathematischen Funktionen des Programms. Dazu gehören zum Beispiel Eigenwerte, Eigenvektoren, der Rang einer Matrix oder das Lösen linearer Gleichungssysteme.

Stolperfallen

Stolperfalle: Fachliche Erweiterungen sollten nicht unüberlegt eingebaut werden. Gerade bei komplexeren Verfahren wie Eigenwertberechnungen ist eine saubere mathematische Grundlage wichtig.

11.2 Programmiertechnische Erweiterungen

Auch die interne Struktur des Programms kann weiter verbessert werden. Eine sinnvolle Erweiterung wäre der Einsatz des **Strategy-Patterns**, um verschiedene Berechnungsverfahren flexibel austauschen zu können. So könnten zum Beispiel unterschiedliche Algorithmen zur Berechnung der Determinante oder der Inversen verwendet werden, ohne die restliche Programmstruktur ändern zu müssen.

Zusätzlich könnte die Validierung der Eingaben weiter ausgebaut werden. Dadurch ließen sich fehlerhafte oder unvollständige Eingaben frühzeitig erkennen und gezielt behandeln. Auch eine stärkere Trennung einzelner Berechnungsschritte würde die Wartbarkeit des Codes verbessern.

Merkkasten

Merke: Programmiertechnische Erweiterungen verbessern nicht die Mathematik selbst, sondern die Struktur, Wartbarkeit und Flexibilität des Codes. Besonders Entwurfsmuster wie das **Strategy-Pattern** sind dafür geeignet.

Stolperfallen

Stolperfalle: Neue Funktionen sollten nicht direkt in bestehende Methoden hineingeschrieben werden, wenn dadurch der Code unübersichtlich wird. Besser ist eine saubere Trennung in klar definierte Bausteine.

11.3 Oberflächenanpassungen

Auch die grafische Benutzeroberfläche kann noch weiter verbessert werden. Dazu gehören zum Beispiel farbliche Anpassungen, größere Abstände zwischen den Elementen, Rahmen oder Icons. Solche Änderungen können die Anwendung moderner und übersichtlicher wirken lassen.

Darüber hinaus könnten die Schaltflächen optisch hervorgehoben oder in einer anderen Form gestaltet werden, etwa mit abgerundeten oder ovalen Buttons. Auch eine tabellarische Eingabe der Matrizen wäre denkbar, da sie die mathematische Struktur besser sichtbar machen würde als einfache Textfelder.

Merkkasten

Merke: Oberflächenanpassungen verbessern vor allem die Bedienbarkeit und die optische Wirkung der Anwendung. Eine gut gestaltete Oberfläche macht das Programm intuitiver und benutzerfreundlicher.

Stolperfallen

Stolperfalle: Eine schöne Oberfläche ersetzt keine klare Struktur. Design sollte die Bedienung unterstützen und nicht von den eigentlichen Funktionen ablenken.

12 Fazit

Im Rahmen dieses Projekts wurde ein **Matrizenrechner in Java** konzipiert und in einer klaren **MVC-Struktur** umgesetzt. Dabei wurden die fachlichen Anforderungen an das Programm systematisch berücksichtigt und die wichtigsten Operationen auf Matrizen implementiert. Dazu

gehören unter anderem die Addition, Subtraktion, Multiplikation, das Transponieren, die Berechnung der Spur, der Determinante und der Inverse.

Besonders wichtig war die saubere Trennung der Aufgaben zwischen **Model**, **View** und **Controller**. Dadurch bleibt der Code übersichtlich, besser wartbar und leichter erweiterbar. Die Rechenlogik ist im Model gebündelt, die Benutzeroberfläche wird in der View dargestellt und der Controller übernimmt die Verbindung zwischen beiden Bereichen.

Auch die Behandlung von **Exceptions** spielt eine wichtige Rolle, da dadurch fehlerhafte Eingaben und unzulässige Operationen kontrolliert abgefangen werden können. Das erhöht die Stabilität des Programms und sorgt für eine bessere Benutzererfahrung.

Insgesamt zeigt das Projekt, wie sich mathematische Inhalte, objektorientierte Programmierung und grafische Benutzeroberflächen sinnvoll miteinander verbinden lassen. Gleichzeitig bietet der Matrizenrechner eine gute Grundlage für weitere Erweiterungen und Verbesserungen.

Darüber hinaus war das Projekt auch fachlich und programmiertechnisch ein wertvoller Lernprozess. Besonders die Umsetzung des MVC-Prinzips, der Umgang mit Exceptions und die strukturierte Zerlegung mathematischer Probleme in einzelne Methoden haben gezeigt, wie wichtig saubere Architektur und klare Verantwortlichkeiten in der Softwareentwicklung sind.

Merkkasten

Merke: Ein gut strukturiertes Programm ist nicht nur funktional, sondern auch übersichtlich, wartbar und erweiterbar. Genau das wird durch die Trennung in Model, View und Controller erreicht.

Anhang

Im Anhang sind die vollständigen Java-Klassen des Matrizenrechners aufgeführt. Dort werden die im Hauptteil beschriebenen Konzepte und Methoden in ihrer vollständigen Implementierung dargestellt. Der Anhang dient dazu, den Quellcode übersichtlich auszulagern, damit der Lesefluss des Hauptteils nicht unterbrochen wird.

Die folgenden Klassen sind enthalten:

- `Matrix` als Model-Klasse mit den mathematischen Operationen
- `MatrixException` für die Fehlerbehandlung
- `MatrixController` zur Steuerung der Benutzeraktionen
- `MatrixView` für die grafische Benutzeroberfläche
- `Main` als Einstiegspunkt der Anwendung

So kann der Code bei Bedarf vollständig nachgelesen werden, während die wichtigsten fachlichen und architektonischen Erklärungen im Hauptteil stehen.

A Java-Code

A.1 Matrix-Klasse

```
1  import java.util.Arrays;
2
3  public class Matrix {
4
5      private double[][] data;
6
7      public Matrix(int zeilen, int spalten) {
8          if (zeilen <= 0 || spalten <= 0) {
9              throw new MatrixParseException("Ungültige Matrixgröße");
10         }
11         data = new double[zeilen][spalten];
12     }
13
14     public Matrix(double[][] data) {
15
16         if (data == null || data.length == 0 || data[0].length == 0) {
17             throw new MatrixParseException("Matrix darf nicht leer sein.");
18         }
19
20         int cols = data[0].length;
21
22         for (int i = 1; i < data.length; i++) {
23             if (data[i].length != cols) {
24                 throw new MatrixParseException("Ungültige Matrixform.");
25             }
26         }
27
28         this.data = new double[data.length][cols];
29
30         for (int i = 0; i < data.length; i++) {
31             System.arraycopy(data[i], 0, this.data[i], 0, cols);
32         }
33     }
34
35     public int zeilen() {
```

```

36     return data.length;
37 }
38
39 public int spalten() {
40     return data[0].length;
41 }
42
43 public double get(int i, int j) {
44     return data[i][j];
45 }
46
47 public void set(int i, int j, double value) {
48     data[i][j] = value;
49 }
50
51 public Matrix add(Matrix other) {
52
53     if (zeilen() != other.zeilen() || spalten() != other.spalten()) {
54         throw new MatrixDimensionException("Dimension mismatch");
55     }
56
57     Matrix result = new Matrix(zeilen(), spalten());
58
59     for (int i = 0; i < zeilen(); i++) {
60         for (int j = 0; j < spalten(); j++) {
61             result.set(i, j, this.get(i, j) + other.get(i, j));
62         }
63     }
64
65     return result;
66 }
67
68 public Matrix subtract(Matrix other) {
69
70     if (zeilen() != other.zeilen() || spalten() != other.spalten()) {
71         throw new MatrixDimensionException("Dimension mismatch");
72     }
73
74     Matrix result = new Matrix(zeilen(), spalten());
75
76     for (int i = 0; i < zeilen(); i++) {
77         for (int j = 0; j < spalten(); j++) {
78             result.set(i, j, this.get(i, j) - other.get(i, j));
79         }
80     }
81
82     return result;
83 }
84
85 public Matrix multiply(Matrix other) {
86
87     if (this.spalten() != other.zeilen()) {
88         throw new MatrixDimensionException("Dimension mismatch");
89     }
90
91     Matrix result = new Matrix(this.zeilen(), other.spalten());
92
93     for (int i = 0; i < this.zeilen(); i++) {
94         for (int j = 0; j < other.spalten(); j++) {
95
96             double sum = 0;

```

```

97         for (int k = 0; k < this.spalten(); k++) {
98             sum += this.get(i, k) * other.get(k, j);
99         }
100     }
101
102     result.set(i, j, sum);
103 }
104 }
105
106 return result;
107 }
108
109 public double trace() {
110     if (zeilen() != spalten()) {
111         throw new MatrixDimensionException("Matrix muss quadratisch sein")
112         ;
113     }
114     double spur = 0;
115     for (int i = 0; i < zeilen(); i++) {
116         spur += this.get(i, i);
117     }
118     return spur;
119 }
120
121 public Matrix transpose() {
122     Matrix transponierte = new Matrix(spalten(), zeilen());
123     for (int i = 0; i < spalten(); i++) {
124         for (int j = 0; j < zeilen(); j++) {
125             transponierte.set(i, j, this.get(j, i));
126         }
127     }
128     return transponierte;
129 }
130
131 public Matrix scalarMultiply(double scalar) {
132     Matrix result = new Matrix(zeilen(), spalten());
133     for (int i = 0; i < zeilen(); i++) {
134         for (int j = 0; j < spalten(); j++) {
135             result.set(i, j, scalar * this.get(i, j));
136         }
137     }
138     return result;
139 }
140
141 public double determinant() {
142     if (this.zeilen() != this.spalten()) {
143         throw new MatrixDimensionException("Die Determinante ist nur fuer
144             quadratische Matrizen definiert.");
145     }
146
147     if (this.zeilen() == 1) {
148         return this.get(0, 0);
149     }
150
151     if (this.zeilen() == 2) {
152         return this.get(0, 0) * this.get(1, 1) - this.get(0, 1) * this.get
153             (1, 0);
154     }
155
156     double summe = 0.0;
157     for (int j = 0; j < this.spalten(); j++) {

```



```

155     double vorzeichen = sign(0, j);
156     summe += vorzeichen * this.get(0, j) * subMatrix(0, j).determinant
157         ();
158 }
159 return summe;
160 }
161 public Matrix inverse() {
162
163     if (zeilen() != spalten()) {
164         throw new MatrixDimensionException("Die Inverse ist nur fuer
165             quadratische Matrizen definiert.");
166     }
167
168     double det = determinant();
169
170     if (det == 0) {
171         throw new MatrixNotInvertibleException("Die Matrix ist nicht
172             invertierbar, da die Determinante 0 ist.");
173     }
174
175     if (zeilen() == 1) {
176         Matrix result = new Matrix(1, 1);
177         result.set(0, 0, 1.0 / get(0, 0));
178         return result;
179     }
180
181     return adjugate().scalarMultiply(1.0 / det);
182 }
183 private Matrix subMatrix(int zeile, int spalte) {
184     Matrix result = new Matrix(this.zeilen() - 1, this.spalten() - 1);
185
186     int neueZeile = 0;
187     for (int i = 0; i < this.zeilen(); i++) {
188         if (i == zeile) {
189             continue;
190         }
191
192         int neueSpalte = 0;
193         for (int j = 0; j < this.spalten(); j++) {
194             if (j == spalte) {
195                 continue;
196             }
197
198             result.set(neueZeile, neueSpalte, this.get(i, j));
199             neueSpalte++;
200         }
201         neueZeile++;
202     }
203
204     return result;
205 }
206
207 private double sign(int row, int col) {
208     return ((row + col) % 2 == 0) ? 1.0 : -1.0;
209 }
210
211 private Matrix adjugate() {
212     int n = this.zeilen();

```

```

213     Matrix kofaktoren = new Matrix(n, n);
214
215     for (int i = 0; i < n; i++) {
216         for (int j = 0; j < n; j++) {
217             Matrix minor = subMatrix(i, j);
218             double vorzeichen = sign(i, j);
219             kofaktoren.set(i, j, vorzeichen * minor.determinant());
220         }
221     }
222
223     return kofaktoren.transpose();
224 }
225
226 @Override
227 public String toString() {
228
229     StringBuilder sb = new StringBuilder();
230
231     for (int i = 0; i < zeilen(); i++) {
232         for (int j = 0; j < spalten(); j++) {
233             sb.append(String.format("%10.2f", this.get(i, j)));
234         }
235         sb.append("\n");
236     }
237     return sb.toString();
238 }
239
240 @Override
241 public int hashCode() {
242     final int prime = 31;
243     int result = 1;
244     result = prime * result + Arrays.deepHashCode(data);
245     return result;
246 }
247
248 @Override
249 public boolean equals(Object obj) {
250     if (this == obj)
251         return true;
252     if (obj == null)
253         return false;
254     if (getClass() != obj.getClass())
255         return false;
256     Matrix other = (Matrix) obj;
257     return Arrays.deepEquals(data, other.data);
258 }
259
260 }

```

A.2 MatrixException

```

1 public class MatrixException extends RuntimeException {
2
3     public MatrixException(String nachricht) {
4         super(nachricht);
5     }
6 }
7 public class MatrixParseException extends MatrixException {
8     public MatrixParseException(String message) {

```

```

9         super(message);
10    }
11 }
12 public class MatrixNotInvertibleException extends MatrixException {
13     public MatrixNotInvertibleException(String message) {
14         super(message);
15     }
16 }
17 public class MatrixDimensionException extends MatrixException {
18     public MatrixDimensionException(String message) {
19         super(message);
20     }
21 }

```

A.3 Controller

```

1 import java.awt.event.ActionEvent;
2 import java.awt.event.ActionListener;
3
4 public class MatrixController {
5
6     private final MatrixView view;
7
8     public MatrixController(MatrixView view) {
9         this.view = view;
10
11         this.view.setAddActionListener(e -> handleAdd());
12         this.view.setSubtractActionListener(e -> handleSubtract());
13         this.view.setMultiplyActionListener(e -> handleMultiply());
14         this.view.setTransposeActionListener(e -> handleTranspose());
15         this.view.setTraceActionListener(e -> handleTrace());
16         this.view.setDeterminantActionListener(e -> handleDeterminant());
17         this.view.setInverseActionListener(e -> handleInverse());
18     }
19
20     private void handleAdd() {
21         try {
22             Matrix a = view.getMatrixA();
23             Matrix b = view.getMatrixB();
24             Matrix result = a.add(b);
25             view.showResult(result.toString());
26         } catch (MatrixException ex) {
27             view.showError(ex.getMessage());
28         }
29     }
30
31     private void handleSubtract() {
32         try {
33             Matrix a = view.getMatrixA();
34             Matrix b = view.getMatrixB();
35             Matrix result = a.subtract(b);
36             view.showResult(result.toString());
37         } catch (MatrixException ex) {
38             view.showError(ex.getMessage());
39         }
40     }
41
42     private void handleMultiply() {
43         try {

```

```

44     Matrix a = view.getMatrixA();
45     Matrix b = view.getMatrixB();
46     Matrix result = a.multiply(b);
47     view.showResult(result.toString());
48 } catch (MatrixException ex) {
49     view.showError(ex.getMessage());
50 }
51 }
52
53 private void handleTranspose() {
54     try {
55         Matrix a = view.getMatrixA();
56         Matrix result = a.transpose();
57         view.showResult(result.toString());
58     } catch (MatrixException ex) {
59         view.showError(ex.getMessage());
60     }
61 }
62
63 private void handleTrace() {
64     try {
65         Matrix a = view.getMatrixA();
66         double result = a.trace();
67         view.showResult(String.valueOf(result));
68     } catch (MatrixException ex) {
69         view.showError(ex.getMessage());
70     }
71 }
72
73 private void handleDeterminant() {
74     try {
75         Matrix a = view.getMatrixA();
76         double result = a.determinant();
77         view.showResult(String.valueOf(result));
78     } catch (MatrixException ex) {
79         view.showError(ex.getMessage());
80     }
81 }
82
83 private void handleInverse() {
84     try {
85         Matrix a = view.getMatrixA();
86         Matrix result = a.inverse();
87         view.showResult(result.toString());
88     } catch (MatrixException ex) {
89         view.showError(ex.getMessage());
90     }
91 }
92 }

```

A.4 View

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionListener;
4
5 public class MatrixView extends JFrame {
6
7     private final JTextArea matrixATextArea;

```

```

8 private final JTextArea matrixBTextArea;
9 private final JTextArea resultTextArea;
10
11 private final JButton addButton;
12 private final JButton subtractButton;
13 private final JButton multiplyButton;
14 private final JButton transposeButton;
15 private final JButton traceButton;
16 private final JButton determinantButton;
17 private final JButton inverseButton;
18
19 public MatrixView() {
20     setTitle("Matrizenrechner");
21     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
22     setSize(900, 600);
23     setLocationRelativeTo(null);
24
25     matrixATextArea = new JTextArea(8, 20);
26     matrixBTextArea = new JTextArea(8, 20);
27     resultTextArea = new JTextArea(10, 30);
28     resultTextArea.setEditable(false);
29
30     addButton = new JButton("+");
31     subtractButton = new JButton("-");
32     multiplyButton = new JButton("*");
33     transposeButton = new JButton("T");
34     traceButton = new JButton("Spur");
35     determinantButton = new JButton("Determinante");
36     inverseButton = new JButton("Inverse");
37
38     JPanel inputPanel = new JPanel(new GridLayout(1, 2, 10, 10));
39     inputPanel.add(new JScrollPane(matrixATextArea));
40     inputPanel.add(new JScrollPane(matrixBTextArea));
41
42     JPanel buttonPanel = new JPanel(new GridLayout(2, 4, 10, 10));
43     buttonPanel.add(addButton);
44     buttonPanel.add(subtractButton);
45     buttonPanel.add(multiplyButton);
46     buttonPanel.add(transposeButton);
47     buttonPanel.add(traceButton);
48     buttonPanel.add(determinantButton);
49     buttonPanel.add(inverseButton);
50
51     JPanel resultPanel = new JPanel(new BorderLayout());
52     resultPanel.add(new JLabel("Ergebnis:"), BorderLayout.NORTH);
53     resultPanel.add(new JScrollPane(resultTextArea), BorderLayout.CENTER);
54
55     setLayout(new BorderLayout(10, 10));
56     add(inputPanel, BorderLayout.NORTH);
57     add(buttonPanel, BorderLayout.CENTER);
58     add(resultPanel, BorderLayout.SOUTH);
59 }
60
61 public Matrix getMatrixA() {
62     return parseMatrix(matrixATextArea.getText());
63 }
64
65 public Matrix getMatrixB() {
66     return parseMatrix(matrixBTextArea.getText());
67 }

```

```

68
69 public void showResult(String text) {
70     resultTextArea.setText(text);
71 }
72
73 public void showError(String message) {
74     JOptionPane.showMessageDialog(this, message, "Fehler", JOptionPane.
75         ERROR_MESSAGE);
76 }
77
78 public void setAddActionListener(ActionListener listener) {
79     addButton.addActionListener(listener);
80 }
81
82 public void setSubtractActionListener(ActionListener listener) {
83     subtractButton.addActionListener(listener);
84 }
85
86 public void setMultiplyActionListener(ActionListener listener) {
87     multiplyButton.addActionListener(listener);
88 }
89
90 public void setTransposeActionListener(ActionListener listener) {
91     transposeButton.addActionListener(listener);
92 }
93
94 public void setTraceActionListener(ActionListener listener) {
95     traceButton.addActionListener(listener);
96 }
97
98 public void setDeterminantActionListener(ActionListener listener) {
99     determinantButton.addActionListener(listener);
100 }
101
102 public void setInverseActionListener(ActionListener listener) {
103     inverseButton.addActionListener(listener);
104 }
105
106 private Matrix parseMatrix(String input) {
107     String[] rows = input.trim().split("\\n");
108     int zeilen = rows.length;
109     String[] firstRow = rows[0].trim().split("\\s+");
110     int spalten = firstRow.length;
111
112     double[][] werte = new double[zeilen][spalten];
113
114     for (int i = 0; i < zeilen; i++) {
115         String[] values = rows[i].trim().split("\\s+");
116         for (int j = 0; j < spalten; j++) {
117             werte[i][j] = Double.parseDouble(values[j].replace(",", "."));
118         }
119     }
120
121     return new Matrix(werte);
122 }

```

A.5 Main-Klasse

```
1 import javax.swing.SwingUtilities;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         SwingUtilities.invokeLater(() -> {
7             MatrixView view = new MatrixView();
8             new MatrixController(view);
9             view.setVisible(true);
10        });
11    }
12 }
```

Musterantworten

1. Warum haben Sie die View nicht komplett frei von Logik gehalten?

Die View enthält nur eine kleine Menge Logik zum Parsen der Benutzereingabe, damit aus dem Textfeld überhaupt ein Matrixobjekt erzeugt werden kann. Die eigentliche Rechenlogik bleibt im Model, und die Aktionen werden über den Controller ausgelöst. Streng nach MVC könnte man das Parsen auch in den Controller verlagern, aber für dieses Projekt war die Lösung praktisch und übersichtlich.

2. Warum verwenden Sie für die Determinante ein rekursives Verfahren statt einer effizienteren Methode?

Das rekursive Verfahren ist für kleine Matrizen gut nachvollziehbar und passt zu meinem Projekt, das vor allem als Einstieg in die GUI- und OOP-Programmierung gedacht war. Es zeigt den mathematischen Zusammenhang zwischen Determinante, Untermatrizen und Kofaktoren sehr anschaulich. Für große Matrizen wäre es nicht optimal, aber für meinen Anwendungsfall ist es fachlich passend.

3. Was passiert, wenn der Benutzer ungleich lange Zeilen eingibt?

Dann ist die Eingabe keine gültige Matrix mehr. Der Konstruktor prüft deshalb, ob alle Zeilen die gleiche Länge haben, und wirft sonst eine passende Exception. So wird verhindert, dass eine unregelmäßige Struktur intern gespeichert wird.

4. Warum haben Sie double statt BigDecimal verwendet?

`double` ist für diesen Matrizenrechner einfacher zu verwenden und für die üblichen Rechenoperationen ausreichend. `BigDecimal` wäre zwar genauer, würde den Code aber deutlich komplizierter machen. Für ein Lernprojekt war mir eine gut verständliche und saubere Umsetzung wichtiger als maximale numerische Präzision.

5. Wieso reicht bei der Inversenberechnung nicht einfach die Bedingung „quadratisch“ aus?

Eine Matrix muss zwar quadratisch sein, damit eine Inverse überhaupt definiert sein kann, aber das allein reicht noch nicht aus. Zusätzlich muss die Determinante ungleich null sein. Nur dann ist die Matrix invertierbar; bei Determinante 0 ist sie singulär.

6. Warum haben Sie equals() und hashCode() beide überschrieben?

`equals()` dient dazu, Matrizen inhaltlich zu vergleichen, also nicht nur die Referenz, sondern die tatsächlichen Werte. `hashCode()` muss dazu passend überschrieben werden, damit gleichartige Matrizen in hashbasierten Datenstrukturen korrekt funktionieren. Beide Methoden gehören in Java zusammen.

7. Welche Nachteile hat Ihre aktuelle Lösung bei sehr großen Matrizen?

Die rekursive Berechnung von Determinante und Inverse ist sehr aufwendig und wächst schnell im Aufwand. Für große Matrizen ist das Verfahren daher deutlich langsamer als effizientere Algorithmen. Für mein Projekt war das aber akzeptabel, weil der Fokus auf Verständlichkeit und Umsetzung lag.

8. Warum haben Sie die Exception-Hierarchie so aufgebaut und nicht alle Fehler direkt mit einer einzigen Exception behandelt?

Mit der Hierarchie kann ich Fehler fachlich genauer unterscheiden, zum Beispiel Eingabefehler, Dimensionsfehler oder Nicht-Invertierbarkeit. Gleichzeitig kann der Controller trotzdem alle Matrixfehler gemeinsam über `MatrixException` abfangen. Das macht die Fehlerbehandlung flexibel und übersichtlich.

9. Warum haben Sie die Matrix intern kopiert statt das Array direkt zu übernehmen?

Damit der innere Zustand der Matrix geschützt bleibt. Wenn ich das Array direkt übernehmen würde, könnte der Aufrufer es später von außen verändern und damit auch die Matrix ungewollt verändern. Die defensive Kopie sorgt also für Kapselung und Unabhängigkeit.

10. Warum haben Sie `toString()` überschrieben?

Damit Matrizen lesbar ausgegeben werden können und nicht nur eine Standard-Objektreferenz erscheint. Das ist wichtig für die Anzeige im Ergebnisfeld und für Debug-Ausgaben.

11. Warum haben Sie bei der Eingabe Kommas als Dezimaltrennzeichen zugelassen?

In Deutschland ist das Komma als Dezimaltrennzeichen üblich. Durch die Ersetzung von Komma durch Punkt vor dem Parsen wird die Eingabe für den Benutzer einfacher und natürlicher.

12. Warum haben Sie die Methode `scalarMultiply()` implementiert?

Die Methode `scalarMultiply()` ist hilfreich, um eine Matrix mit einem Skalar zu multiplizieren, zum Beispiel bei der Berechnung der Inversen. Außerdem macht sie den Code übersichtlicher, weil dieser Rechenschritt an einer Stelle gekapselt ist.

13. Warum gibt es eine eigene Methode `trace()` für die Spur?

Die Spur ist eine wichtige mathematische Eigenschaft einer quadratischen Matrix. Mit einer eigenen Methode lässt sich diese Berechnung klar vom Rest der Rechenlogik trennen.

14. Warum haben Sie bei der Multiplikation die Bedingung `this.spalten() == other.zeilen()` geprüft?

Diese Bedingung ist notwendig, damit die Matrixmultiplikation mathematisch definiert ist. Nur wenn die Anzahl der Spalten der ersten Matrix zur Anzahl der Zeilen der zweiten Matrix passt, kann das Produkt berechnet werden.

15. Warum haben Sie für die Ausgabe `StringBuilder` statt String-Verkettung verwendet?

`StringBuilder` ist bei längeren Ausgaben effizienter, weil nicht bei jeder Verkettung ein neuer String erzeugt werden muss. Das ist besonders bei der zeilenweisen Ausgabe einer Matrix sinnvoll.

16. Warum ist die Methode `subMatrix()` privat?

Die Methode wird nur intern für die Berechnung von Determinante und Inverse benötigt. Deshalb muss sie nicht von außen aufgerufen werden und bleibt als Hilfsmethode gekapselt.

17. Warum haben Sie die Methode `transpose()` implementiert?

Die Transposition ist eine grundlegende Matrixoperation und wird außerdem bei der Berechnung der Adjunkte benötigt. Daher ist sie sowohl fachlich als auch technisch eine sinnvolle eigene Methode.

18. Warum werfen Sie bei ungültigen Operationen `Exceptions` statt `null` zurückzugeben?

Exceptions machen Fehler klar sichtbar und verhindern, dass fehlerhafte Ergebnisse stillschweigend weiterverwendet werden. `null` wäre hier unklarer und könnte später zu schwer auffindbaren Fehlern führen.

19. Warum haben Sie die View mit Swing umgesetzt?

Swing eignet sich gut für eine einfache grafische Benutzeroberfläche in Java. Es ist direkt verfügbar und ermöglicht die Umsetzung von Eingabefeldern, Buttons und Ausgabebereichen mit relativ wenig Aufwand.

20. Warum haben Sie die Oberfläche in mehrere Bereiche aufgeteilt?

Die Aufteilung in Eingabe, Aktionen und Ausgabe macht die Bedienung übersichtlich. Der Benutzer erkennt dadurch schneller, wo er Werte eingibt, welche Operationen möglich sind und wo das Ergebnis erscheint.

21. Warum haben Sie die Determinante rekursiv statt mit einem effizienteren Verfahren berechnet?

Die rekursive Berechnung ist für kleine Matrizen gut nachvollziehbar und zeigt den mathematischen Zusammenhang zwischen Untermatrizen, Kofaktoren und Determinante sehr anschaulich. Für mein Projekt stand die verständliche Umsetzung im Vordergrund, nicht die optimale Laufzeit.

22. Welche Auswirkungen hat die Verwendung von `double` auf die Genauigkeit Ihrer Berechnungen?

`double` kann bei bestimmten Rechnungen zu Rundungsfehlern führen, weil Gleitkommazahlen nicht immer exakt dargestellt werden können. Für den Matrizenrechner war das aber akzeptabel, weil die Implementierung einfach und gut verständlich bleiben sollte.

23. Warum ist die Methode `inverse()` auf die Adjunkte angewiesen?

Die Inverse wird über die Formel $A^{-1} = \frac{1}{\det(A)} \cdot \text{adj}(A)$ berechnet. Dafür braucht man zuerst die Adjunkte, die aus der Cofaktormatrix durch Transposition entsteht.

24. Welche Rolle spielt die Methode `sign()` bei der Berechnung der Determinante und der Adjunkte?

Die Methode `sign()` liefert das Vorzeichen des jeweiligen Kofaktors nach dem Schachbrettmuster. Dadurch wird das Vorzeichen zentral berechnet und nicht an mehreren Stellen im Code wiederholt.

25. Warum haben Sie bei der Matrixaddition und -subtraktion neue Matrizen erzeugt statt das Original zu verändern?

So bleibt der ursprüngliche Zustand der Eingabematrizen erhalten. Das ist übersichtlicher, vermeidet Seiteneffekte und passt gut zum Prinzip der Kapselung.

26. Warum ist die Methode `subMatrix()` für die Inversenberechnung unverzichtbar?

Für die Berechnung der Kofaktoren muss jeweils eine Zeile und eine Spalte entfernt werden. Genau das erledigt `subMatrix()`, indem sie die passende Untermatrix erzeugt.

27. Warum haben Sie die Fehlerbehandlung über Exceptions und nicht über Rückgabewerte gelöst?

Exceptions trennen den normalen Programmfluss von Fehlerfällen klar voneinander. Dadurch ist der Code lesbarer und Fehler können zentral im Controller abgefangen werden.

28. Welche Vorteile hat die Trennung von View und Model für die Testbarkeit?

Das Model kann unabhängig von der Benutzeroberfläche getestet werden. Dadurch lassen sich die mathematischen Funktionen prüfen, ohne die GUI starten zu müssen.

29. Warum haben Sie die Methode `equals()` mit `Arrays.deepEquals()` umgesetzt?

Weil die Matrix intern aus einem zweidimensionalen Array besteht. `Arrays.deepEquals()` vergleicht den Inhalt dieser verschachtelten Struktur korrekt und nicht nur die Referenz.

30. Warum ist Ihre Lösung für große Matrizen nicht optimal, aber trotzdem sinnvoll?

Für große Matrizen ist die rekursive Berechnung sehr langsam, aber für mein Projekt war die Nachvollziehbarkeit wichtiger als maximale Effizienz. Als Lernprojekt zeigt die Lösung die mathematischen Zusammenhänge und die Architektur des Programms gut.

31. Warum ist Ihre Implementierung der Inversen nicht für jede Matrix geeignet?

Die Implementierung setzt voraus, dass die Matrix quadratisch und ihre Determinante ungleich null ist. Für nicht quadratische oder singuläre Matrizen ist eine Inverse mathematisch nicht definiert.

32. Warum reicht es nicht, in der View nur die Eingabe zu prüfen?

Die View kann nur einfache Eingaben kontrollieren, aber die mathematische Gültigkeit muss im Model geprüft werden. Dort liegen die fachlichen Regeln, zum Beispiel für Dimensionen oder Invertierbarkeit.

33. Was passiert, wenn der Benutzer eine leere Zeile eingibt?

Dann kann die Eingabe keine gültige Matrix mehr darstellen. Solche Fälle müssen abgefangen werden, damit keine ungültigen Arrays erzeugt werden.

34. Warum ist Ihre rekursive Determinantenberechnung bei größeren Matrizen problematisch?

Weil sich die Anzahl der Teilprobleme sehr schnell erhöht. Dadurch wächst der Aufwand stark an, weshalb das Verfahren für große Matrizen deutlich langsamer wird.

35. Warum haben Sie nicht direkt eine fertige Bibliothek für Matrizen verwendet?

Weil das Ziel des Projekts das eigene Verstehen und Umsetzen der mathematischen und objekt-orientierten Grundlagen war. Eine Bibliothek hätte diesen Lernaspekt deutlich reduziert.

36. Warum ist es wichtig, dass ‘equals()’ und ‘hashCode()’ zusammenpassen?

Weil hashbasierte Datenstrukturen auf beiden Methoden aufbauen. Wenn zwei Objekte als gleich gelten, müssen sie auch denselben Hash-Wert haben, sonst entstehen Fehler bei der Speicherung und Suche.

37. Warum ist ‘subMatrix()’ privat und nicht öffentlich?

Weil die Methode nur intern für Determinante und Inverse gebraucht wird. Eine öffentliche Sichtbarkeit würde unnötig viele Zugriffe erlauben und die Kapselung schwächen.

38. Warum ist ‘double’ bei der Inversenberechnung nicht immer exakt?

Weil Gleitkommazahlen nur begrenzt genau dargestellt werden können. Dadurch können bei Berechnungen kleine Rundungsfehler entstehen.

39. Warum kann man die Inverse nicht einfach durch Division berechnen?

Eine Matrixdivision ist mathematisch nicht direkt definiert. Stattdessen wird die Inverse über Determinante und Adjunkte berechnet und anschließend mit der Matrix multipliziert.

40. Warum ist Ihre Lösung zwar korrekt, aber nicht die effizienteste?

Weil sie auf Verständlichkeit und Nachvollziehbarkeit ausgelegt ist. Für ein Lernprojekt ist es wichtiger, dass die mathematischen Zusammenhänge klar umgesetzt werden, als dass die Laufzeit optimal ist.